Safe Interactions via Monte Carlo Linear-Quadratic Games

BENJAMIN A. CHRISTIE and DYLAN P. LOSEY, Virginia Tech, USA

Safety is critical during human-robot interaction. But — because people are inherently unpredictable — it is often difficult for robots to plan safe behaviors. Instead of relying on our ability to anticipate humans, here we identify robot policies that are robust to unexpected human decisions. We achieve this by formulating human-robot interaction as a zero-sum game, where (in the worst case) the human's actions directly conflict with the robot's objective. Solving for the upper value of this game provides robot policies that combine safety and performance across a wide range of human actions. Existing approaches attempt to find these optimal policies by leveraging Hamilton-Jacobi analysis (which is often intractable) or linear-quadratic approximations (which are often inexact). By contrast, in this work we propose a computationally efficient and theoretically supported method that approaches a robust, Stackelberg-style policy. Our approach (which we call *MCLQ*) leverages linear-quadratic games to obtain an initial guess at safe robot behavior, and then iteratively refines that guess with a Monte Carlo search. Not only does MCLQ provide real-time safety adjustments, but it also enables the designer to tune how conservative the robot is — preventing the system from focusing on unrealistic human behaviors. Simulations in point-mass, driving, and manipulation environments, together with an in-person user study in which participants worked alongside an aerial drone, suggest that this approach improves collision avoidance in human-robot interaction while reducing computation time.

CCS Concepts: • **Computing methodologies** → *Robotic planning*; **Planning under uncertainty**.

Additional Key Words and Phrases: Human-Robot Interaction, Safety

1 INTRODUCTION

Interacting with people is challenging because humans are inherently unpredictable. Consider Figure 1 where an autonomous drone is flying near a human worker. This robot has some high-level task it wants to complete (e.g., environment monitoring), as well as a low-level controller which dictates how the robot should accomplish this task (e.g., circling the room). If the robot knew precisely what the human was going to do, it could anticipate the human's actions and choose behaviors that maintain a safe distance between agents. But because people often take unexpected actions, real human behavior will deviate from the robot's model. As a result, the robot's original plan — which it thought was safe — may actually be unsafe. Returning to our example, a drone that turns left (because it predicts the human will stop) actually puts both agents in danger (because the human unexpectedly keeps walking forwards).

To address this problem, today's robots recognize that they are often uncertain about the human's actions. Instead of assuming the human will follow an exact model, these robots search for plans that are safe across a distribution of human behaviors. There are two general approaches here. The first is a *precise* method: the robot reasons over the distribution of possible human trajectories, and then chooses the optimal control policy that maximizes safety across the distribution [8, 18, 22]. The second approach is based on *approximations*: the robot simplifies the system dynamics and objectives, and then obtains a closed-form policy that maximizes safety under the simplified setting [20, 45]. Unfortunately, both types of approaches have significant limitations. Theoretically exact solutions are often computationally intractable; e.g., our autonomous vehicle could not apply these methods in real-time to adjust its behaviors. On the other hand, approximate solutions are imprecise, and may result in unsafe behaviors that are *locally* optimal but globally unsafe as the system becomes increasingly complex and nonlinear.

This work is supported in part by NSF Grant #2129201.
Authors' address: Benjamin A. Christie, benc00@vt.edu; Dylan P. Losey, losey@vt.edu, Virginia Tech, Department of Mechanical Engineering, 635 Prices Fork Rd, Blacksburg, VA, 24060, USA.

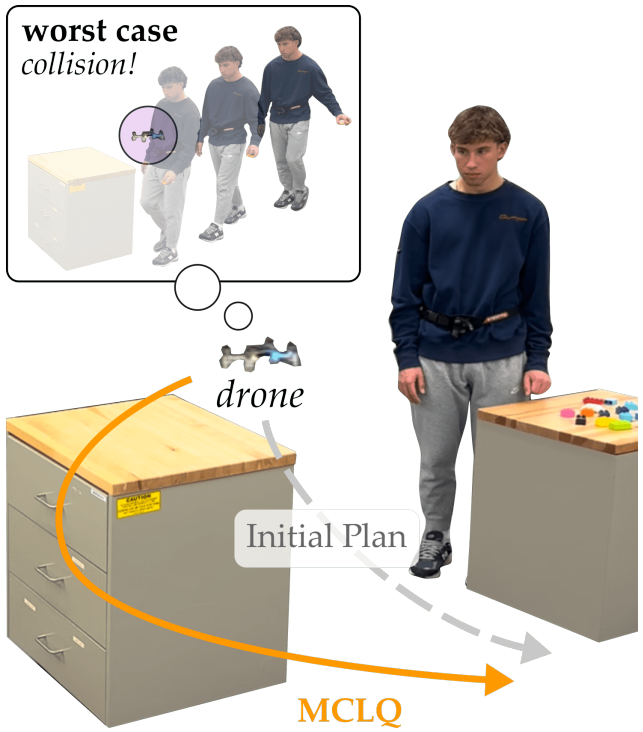


Fig. 1. Human and drone moving in a shared workspace. Under our proposed method (MCLQ), the drone reasons about worst case human actions within designer-specified bounds, and then selects safe behaviors in response to those actions. For instance, here the drone moves across the table to prevent a potential collision.

In this paper we propose an action-augmentation method for human-robot interaction that robots can leverage to tune their real-time behavior and enhance safety around unpredictable humans. Our proposed method achieves precise and tractable performance by combining both types of prior approaches. Specifically, our insight is that:

We can leverage approximate methods to seed safe robot behavior, and then apply a stochastic local search to guide that initial guess towards optimal solutions.

We show an example of our method in Figure 1. The robot approximates the interaction as a linear-quadratic (LQ) system, and finds an action sequence that maximizes robot performance under the worst case human response. In parallel, the robot refines this action sequence using sampling methods that noisily converge towards the open-loop optima of a zero-sum game while stochastically escaping local optima. Robots that execute the resulting actions are robust to unpredictable humans within designer-specified bounds: if the person suddenly changes direction within these bounds, the autonomous drone has already accounted for that change during planning.

Overall, we make the following contributions:

Identifying Robust Behavior. To maintain safety despite unpredictable humans, we frame interaction as a zero-sum game. Here the robot tries to minimize its cost while realizing an adversarial human might take actions to maximize that cost. The game's upper value defines a robot policy that is robust to unexpected human behavior; we develop a method (MCLQ) to tractably approximate this robust policy.

Uniting Prior Works. Our approach to find robust robot policies blends aspects of approximate and precise methods. Specifically, we leverage LQ games to get an initial action sequence, and then refine that sequence with Metropolis-Hastings sampling. Our analysis bounds the probability that MCLQ returns a trajectory close to the robust robot policy, and shows how this bound depends on the parameters chosen by the designer.

Adjusting Safety Margins. Our MCLQ framework enables practical advantages that go beyond existing methods. For example, designers can constrain the safety margin, i.e., the range of human actions the robot reasons over. Decreasing this safety margin causes the robot to rely more on its human predictions, preventing the robot from focusing on unrealistic human behaviors and becoming overly conservative.

Conducting Experiments. We conduct simulations and a user study to compare our approach to state-of-the-art methods. In simulation, we show that MCLQ solves for safe control policies more rapidly than HJB and iLQ methods, while also achieving higher performance than existing iLQ approximations. In an in-person user study with an aerial drone, MCLQ led to fewer collisions and more completed task revolutions than an iLQ baseline, and participants rated the MCLQ drone as safer and more attentive.

2 RELATED WORK

Related research has introduced a variety of control algorithms to ensure safety during human-robot interaction [22]. Within this larger field, our approach is most connected to game-theoretic controllers. These methods frame the human and robot as two agents with their own objectives, and solve for robot policies that are robust (i.e., minimize worst-case costs) while accounting for the human's decisions.

Exact Methods. Game-theoretic works treat human-robot interaction as interconnected system: both agents have a task they are trying to accomplish, and the actions of each agent affect one another. We can formalize these interconnected dynamical games with Hamilton-Jacobi equations [10, 48]. Precisely solving the Hamilton-Jacobi equations provides the optimal, game-theoretic behavior for each agent; e.g., a policy for the autonomous car that avoids collisions with the human. Hamilton-Jacobi analysis has therefore become a gold standard for safe interaction [6–9, 19]. Unfortunately, most Hamilton-Jacobi equations have no analytical solution, and numerical methods are often intractable — requiring both exponential time and computational memory [20, 44]. As such, within this paper we develop a real-time approximation that converges towards the ideal solution provided by the Hamilton-Jacobi framework.

Linear-Quadratic Approximations. We are not the first to try to approximate the game-theoretic solution. For example, [9, 18, 27, 34, 35] propose neural network models, and [13, 20, 28] outline open-loop methods. One particularly promising approximation recognizes that we can analytically solve the Hamilton-Jacobi equations in a special case: if the system dynamics are linear and the costs are quadratic [10]. For these linear-quadratic (LQ) games the optimal robot policy is closely connected to linear-quadratic regulators [4, 52]. Robots can solve LQ games quickly, providing a tractable method to obtain safe behaviors that match the gold standard of Hamilton-Jacobi analysis. Indeed — even if the system is not linear-quadratic — recent works have tried to apply sequential linear-quadratic (iLQ) approximations [20, 31]. Under these iLQ approaches the robot iteratively linearizes the dynamics and quadraticizes the cost before solving the resulting LQ game to find the human and robot control gains. On the one hand, iLQ provides an increasingly prevalent method to tractably identify safe controllers. But on the other hand, this approximation is fundamentally limited by its reliance on linear-quadratic systems — as the real interaction diverges from the

simplification, iLQ falls short. Overall, this gap motivates our work towards methods that combine theoretical exactness and practical implementation.

Safety Filters and Risk-Sensitive Control. A complementary line of work enforces safety as a hard constraint. Control barrier functions (CBFs) render a safe set forward-invariant by constraining the robot's control input, typically through a quadratic program solved at each timestep [2, 3]. Model predictive safety certification (MPSC) and related predictive safety filters minimally modify a nominal or learned controller so that a backup trajectory to a safe set always exists [49, 50]. These methods provide formal guarantees, but they require a known safe set, a valid barrier function or backup policy, and a bound on the human's impact on the robot's planned trajectory. Risk-sensitive approaches replace the worst case with a risk measure such as the conditional value-at-risk (CVaR), optimizing the expected cost in the tail of a distribution over outcomes [14, 33, 40]. As we will show, our proposed method plays a related role: supporting a tunable parameter that can interpolate between relying on the nominal human model and reasoning over the full worst case. Finally, conformal prediction provides distribution-free prediction regions for other agents' future motions with guaranteed coverage, which can be used to plan safe robot trajectories in dynamic environments [16, 32]. These calibrated regions are complementary to our proposed method, as they can be used to select bounds on the human's actions from an offline dataset.

Accounting for the Human. Using the game-theoretic solution leads to policies that are robust to worst-case disturbances. However, doing so can make robot policies overly conservative — even when the human is not interfering with the robot's objective. Recent works [20, 23, 24, 36, 41, 45] have attempted to mitigate this issue by using a model of the human that updates over the course of an interaction. The authors of [45] maintain an estimate of the current user's *role* in a Stackleberg-style formulation. If they are a follower, then the robot must be more cautious, whereas if they are a leader, then the robot can find calculate an optimal response. Unfortunately, this approach is limited to settings where a backwards-reachable tube (representing the agents' actions) can be calculated in real-time, such as in linear-quadratic games. Likewise in [23], a shielding-aware dual control method is proposed that has theoretical guarantees on robot safety while maintaining an estimate of the uncertain agent's parameters. This sampling-based approach is similar to our proposed approach, but it is limited to control-affine dynamics and Boltzmann-rational human models. Other works such as [15, 29, 36–38] leverage learned models that adapt to the human and predict their future intent. Although these approaches effectively optimize for performance in the *average* interaction, they are not robust to *worst-case* actions that the human may select. In this work we propose a flexible, real-time approach that accounts for worst-case human actions and determines robot responses which are robust to these actions within designer specified bounds.

3 PROBLEM STATEMENT

We are interested in interactions between a single human and a single robot. The robot has a task to perform, and to complete this task the robot must reason over the human (e.g., avoid colliding with the human worker). Without loss of generality, we assume the robot has an initial control policy for the task, as well as a nominal model for predicting the human's behavior. Our approach will adjust the robot's policy to enhance safety even when the human deviates from the nominal model. To achieve this safety, we consider the worst-case interaction (within designer-specified bounds): i.e., we identify which actions the human can take that will have the worst impact on the robot's performance. Below we formalize this problem setting.

Zero-Sum Game. Let the system have state $x \in \mathcal{X}$ (e.g., the position of both agents). The robot takes action $u \in \mathcal{U}$, and the human takes action $w \in \mathcal{W}$ (e.g., the human and robot velocities). At

each timestep t, the system state transitions according to the deterministic, discrete-time dynamics:

$$x^{t+1} = f(x^t, u^t, w^t) \tag{1}$$

An interaction lasts for a total of T timesteps. The system begins the interaction in state x^0 and during the interaction it follows a trajectory $\xi = \{(x, u, w)^0, \cdots, (x, u, w)^{T-1}, x^T\}$. Since the dynamics are deterministic, we can abbreviate this trajectory as a sequence of robot and human actions $\xi(x^0) = \{(u, w)^0, \cdots, (u, w)^{T-1}\} = (u^{0:T}, w^{0:T})$.

The robot's goal is to select actions that will cause the system state to update in a way that completes the robot's task. More formally, the robot has a cost function it seeks to *minimize* across the interaction:

$$\begin{aligned} J\left(x^t, u^{t:T}, w^{t:T}\right) &= \sum_{\tau=t}^{T-1} j(x^\tau, u^\tau, w^\tau) + D(x^T) \\ \text{s.t. } x^{\tau+1} &= f\left(x^\tau, u^\tau, w^\tau\right) \end{aligned} \tag{2}$$

Here J is the cumulative cost, $j(x, u, w)$ is the cost at a single timestep, and $D(x^T)$ is the bequest state cost. We emphasize that the robot's cost in Equation (2) depends on the human's actions w. For example, the autonomous drone will incur a significant penalty if the human moves into that agent. To optimize cost, the robot has an initial policy $\hat{\pi}_{\mathcal{R}}$ that determines how it will complete the task. The robot may also have some guess for how the human agent will behave: $\hat{\pi}_{\mathcal{H}}$. Both policies are mappings from states x to actions:

$$\hat{\pi}_{\mathcal{R}} : \mathcal{X} \mapsto \mathcal{U} \tag{3}$$

$$\hat{\pi}_{\mathcal{H}} : \mathcal{X} \mapsto \mathcal{W} \tag{4}$$

In practice, the real human will inevitably deviate from the robot's model. A worst-case human selects actions that *maximize* the cost in Equation (2). Formally, in this worst-case the human and robot are participating in a two-player *zero-sum* game [10]: the robot is trying to minimize its cost, whereas the antagonistic human is attempting to maximize that same cost, directly opposing the robot's objective. One benefit of focusing on the worst-case human is that — if the robot's behavior is safe in this worst case — we know it is safe for other human responses.

In this work, we assume that the dynamics f, the stage cost j, and the final cost D have continuous first and second derivatives with respect to the state x and the actions u and w. Section 4.1 uses these derivatives to form an initial LQ approximation, and Section 4.4 uses them to bound how strongly the robot's actions affect the human's incentives. We also assume that the action sets $\mathcal{U}$ and $\mathcal{W}$ are compact and convex (e.g., there are bounds on each agent's velocity). In practice, this assumption is naturally satisfied by the limits of robot actuators and human biomechanics. Together, these two assumptions keep the interaction well behaved: because the interaction lasts a finite number of timesteps T and the actions are bounded, the states the agents can reach, the cost J, and its derivatives are all bounded, and the worst-case human response always exists. Beyond these assumptions we do not restrict the dynamics or the cost. The dynamics may be nonlinear or unstable, and J may be non-convex in the robot's actions and non-concave in the human's actions.

Robust Minimax Formulation and Game Values. Similar to prior works [43, 45, 51], we have formulated human-robot interaction as a zero-sum game. The advantage of this formulation is that identifying robot behavior that is safe in the worst-case ensures safety regardless of the human's actual deviation. To determine what this worst-case behavior is, we turn to the bounds of optimal behavior in zero-sum games. In general, zero-sum games are characterized by their upper and lower values. The lower value of the game represents a scenario where the human must commit to

an action sequence first, allowing the robot to optimally respond:

$$\underline{V}(x) = \sup_{w^{0:T} \in \mathcal{W}^T} \inf_{u^{0:T} \in \mathcal{U}^T} \Big(J(x, u^{0:T}, w^{0:T}) \Big) \tag{5}$$

In this situation, the robot is at an advantage. Note that the inner inf operator is evaluated for a specific, fixed human action $w^{0:T}$. Since the robot can see what the human's action is, it can always choose the best response that minimizes the overall cost J. Conversely, the upper value of the game represents the scenario where the robot commits to a trajectory, and the adversarial human responds with the worst-case optimal sequence:

$$\overline{V}(x) = \inf_{u^{0:T} \in \mathcal{U}^T} \sup_{w^{0:T} \in \mathcal{W}^T} \Big(J(x, u^{0:T}, w^{0:T}) \Big) \tag{6}$$

Here, the robot is at a disadvantage. It cannot observe the human's action: instead, it must plan an action that minimizes the overall cost, regardless of what the human chooses. Since the human — wanting to maximize the robot's cost — can observe the robot's action, they can always choose an action that leads to worst-case performance. A pure-strategy equilibrium point between these two values only exists if the Isaacs' Condition holds (for details, see Sion's Minimax Theorem in [42] and Isaacs' Condition in [26]). However, because our interaction involves complex non-linear dynamics (such as articulated manipulators) and non-quadratic objective costs (such as logarithmic barrier functions), the cost landscape J is generally neither strictly convex in u nor strictly concave in w, which violates Isaacs' condition. Thus, we recognize that there may be an information gap between the two values where $\overline{V}(x) \geq \underline{V}(x)$.

To be robust to the human's actions, the robot must prepare for the worst case: the *upper* value of the game. By planning against $\overline{V}(x)$, the robot identifies a robust Stackelberg-style policy where it acts as the leader, committing to an open-loop plan and anticipating the worst-case best-response from the human follower. Therefore, our objective is to find the robust open-loop robot policy:

$$\pi_{\mathcal{R}} = \arg\inf_{u^{0:T} \in \mathcal{U}^T} \sup_{w^{0:T} \in \mathcal{W}^T} \Big(J(x^t, u^{0:T}, w^{0:T}) \Big) \tag{7}$$

In other words, the robot policy should consider the worst-case action of the human before it selects its own action. The robot policy that maximizes *robustness* follows the upper value of the game. To be clear, this robustness is a *soft* measure of safety. In this work, we focus on safety in terms of *collision avoidance*. We encode safety as a soft penalty in the robot's cost J (e.g., a term that grows as the robot approaches the human), and not as a hard constraint on the system state. Equation (7) then minimizes the worst-case value of this cost over all human actions. Accordingly, our approach does not provide formal guarantees such as forward-invariant sets or reachability certificates. This same formulation applies to other safety criteria that can be expressed as smooth terms in the robot's cost (e.g., keeping a margin from workspace boundaries or limiting speed near the human), but our experiments focus on collision avoidance.

Intractability of the Robust Minimax. Directly solving for the robust policy $\pi_{\mathcal{R}}$ is computationally intractable for continuous, high-dimensional action spaces. While traditional approaches attempt to solve the continuous-time counterpart via the Hamilton-Jacobi-Isaacs (HJI) partial differential equations to find global feedback policies, such methods suffer heavily from the curse of dimensionality and cannot be run in real-time. Instead, other approaches aim to find a *locally* robust policy by approximating the system as linear-quadratic. The information gap that results from this approximation is theoretically 0 and an optimal robot policy can be found in closed form. Unfortunately, since the resulting lower and upper values of this approximation are not the same as the original game, the policy that is maximally robust in the approximate game may not be robust in the original system. Given that the robot has some initial policy $\hat{\pi}_{\mathcal{R}}$, our goal is to develop a

real-time approach that *adjusts* the robot's policy so that it converges towards the upper value of the underlying zero-sum game. Because we cannot tractably identify this ideal policy through Equation (7), we must develop an approximation that the robot can leverage efficiently in real-time.

4 MONTE CARLO LINEAR-QUADRATIC GAMES

In this section we propose our real-time method for safe interactions based on zero-sum games. To approximate $\pi_{\mathcal{R}}$ in real-time, we propose a two-stage approach: (1) an analytical "warm-start" using a Linear-Quadratic (LQ) game approximation, and (2) a dual-loop Metropolis-Hastings search that explores the non-convex landscape to minimize the upper value $\overline{V}$. Our overall approach — Monte Carlo Linear-Quadratic Games (**MCLQ**) — combines rapid LQ solutions with a parallel local search. We theoretically demonstrate that MCLQ improves upon existing LQ baselines and converges towards a robust robot policy. We conclude with the practical advantages of our MCLQ framework, including a designer-specified safety margin that prevents the robot from becoming overly conservative (Section 4.3). An implementation of MCLQ is available here: https://github.com/safe-interactions-mclq/mclq.

4.1 Obtaining an Initial Action Trajectory

Our method attempts to find the sequence of actions $u^{0:T}$ that optimize Equation (7). To get an initial estimate of these actions, we simplify the actual system into an LQ approximation. Starting with f, we linearize the dynamics:

$$f(x,u,w,t) \approx Ax(t) + Bu(t) + Dw(t) \tag{8}$$

and quadraticize the state-action cost function J:

$$J(x,u,w,t) \approx x(t)'Qx(t) + u(t)'R_u u(t) - w(t)'R_w w(t) \tag{9}$$

where $\circ'$ represents transposition and $Q, R_u, R_w \succ 0$. In this localized, quadratic regime, the cost function is strictly convex in u and strictly concave in w. Consequently, the Isaacs' condition holds locally, and a unique feedback Nash Equilibrium exists. We perform this LQ approximation once, at the start of the interaction, about a randomly sampled initial trajectory. Note that this LQ approximation is processed for the entire trajectory across timesteps $\hat{t} \in \{t, \dots, T+t\}$. Put another way, the LQ approximation is performed for each state-action pair in the trajectory. Under this LQ approximation the Nash Equilibrium policies exhibit linear state-feedback behavior [10]. These policies are captured by the gain matrices:

$$\begin{aligned} u(t) &= -K(t)x(t) \\ w(t) &= -L(t)x(t) \end{aligned} \tag{10}$$

Crucially, we do not use these gains for closed-loop control. Instead, we use them to generate an initial open-loop seed by rolling out the linearized dynamics. This seed provides an informed starting point, ensuring that the subsequent stochastic search begins in a region of high dynamic feasability. To find the gain matrices, we employ the discrete-time algebraic Riccati Equation (DARE). For a particular robot gain matrix K, the optimal human gain matrix L can be found in closed form by solving the DARE:

$$\begin{aligned} \boldsymbol{P}_{\boldsymbol{K},\boldsymbol{L}(\boldsymbol{K})} &= \boldsymbol{Q} + \boldsymbol{K}'\boldsymbol{R}_u\boldsymbol{K} + (\boldsymbol{A}-\boldsymbol{B}\boldsymbol{K})'\widetilde{\boldsymbol{P}}_{\boldsymbol{K},\boldsymbol{L}(\boldsymbol{K})}(\boldsymbol{A}-\boldsymbol{B}\boldsymbol{K}) \\ \text{s.t.} \quad & \Re\left(\|\boldsymbol{P}_{\boldsymbol{K},\boldsymbol{L}(\boldsymbol{K})}\|\right) \geq 0 \\ & \Re\left(\|\boldsymbol{R}_w - \boldsymbol{D}'\boldsymbol{P}_{\boldsymbol{K},\boldsymbol{L}(\boldsymbol{K})}\boldsymbol{D}\|\right) > 0 \end{aligned} \tag{11}$$

where $\widetilde{\boldsymbol{P}}_{\boldsymbol{K},\boldsymbol{L}(\boldsymbol{K})}$ is condensed for brevity:

$$\widetilde{\boldsymbol{P}}_{\boldsymbol{K},\boldsymbol{L}(\boldsymbol{K})} \equiv \boldsymbol{P}_{\boldsymbol{K},\boldsymbol{L}(\boldsymbol{K})} + \boldsymbol{P}_{\boldsymbol{K},\boldsymbol{L}(\boldsymbol{K})}\boldsymbol{D} \cdot (\boldsymbol{R}_w - \boldsymbol{D}'\boldsymbol{P}_{\boldsymbol{K},\boldsymbol{L}(\boldsymbol{K})}\boldsymbol{D})^{-1}\boldsymbol{D}'\boldsymbol{P}_{\boldsymbol{K},\boldsymbol{L}(\boldsymbol{K})} \tag{12}$$

and the terminal condition is $\boldsymbol{P}_{\boldsymbol{K},\boldsymbol{L}(\boldsymbol{K})} = \boldsymbol{Q}$. The bolded matrices shown in Equation (11) are the compacted notation presented in [52] with appropriate padding:

$$\boldsymbol{Q} \equiv \text{diag}(Q^0, \ldots, Q^T), \quad \boldsymbol{R_u} \equiv \text{diag}(R_u^0, \ldots, R_u^{T-1}), \quad \boldsymbol{R_w} \equiv \text{diag}(R_w^0, \ldots, R_w^{T-1})$$

$$\boldsymbol{A} \equiv \begin{bmatrix} \mathbf{0} & \mathbf{0} \\ \text{diag}(A^0, \ldots, A^T) & \mathbf{0} \end{bmatrix}, \quad \boldsymbol{B} \equiv \begin{bmatrix} \mathbf{0} \\ \text{diag}(B^0, \ldots, B^{T-1}) \end{bmatrix}, \quad \boldsymbol{D} \equiv \begin{bmatrix} \mathbf{0} \\ \text{diag}(D^0, \ldots, D^{T-1}) \end{bmatrix}$$

$$\boldsymbol{K} \equiv \begin{bmatrix} \text{diag}(K^0, \ldots, K^{T-1}) & \mathbf{0} \end{bmatrix}, \quad \boldsymbol{L} \equiv \begin{bmatrix} \text{diag}(L^0, \ldots, L^{T-1}) & \mathbf{0} \end{bmatrix}$$

The unique human gain matrix $\boldsymbol{L}(\boldsymbol{K})$ that *maximizes* the quadraticized cost is:

$$\boldsymbol{L}(\boldsymbol{K}) = (-\boldsymbol{R}_w + \boldsymbol{D}'\boldsymbol{P}_{\boldsymbol{K},\boldsymbol{L}(\boldsymbol{K})}\boldsymbol{D})^{-1}\boldsymbol{D}'\boldsymbol{P}_{\boldsymbol{K},\boldsymbol{L}(\boldsymbol{K})}(\boldsymbol{A} - \boldsymbol{B}\boldsymbol{K}) \tag{13}$$

Note that $\boldsymbol{L}(\boldsymbol{K})$ is the *worst-case* gain matrix given the robot's policy and objective; we have found the human actions that maximize the robot's cost. To develop the correct response to this worst-case, we use a similar equation to compute the robot gain matrix $\boldsymbol{K}(\boldsymbol{L})$:

$$\boldsymbol{K}(\boldsymbol{L}) = (\boldsymbol{R}_u + \boldsymbol{B}'\boldsymbol{P}_{\boldsymbol{K}(\boldsymbol{L}),\boldsymbol{L}}\boldsymbol{B})^{-1}\boldsymbol{B}'\boldsymbol{P}_{\boldsymbol{K}(\boldsymbol{L}),\boldsymbol{L}}\,(\boldsymbol{A} - \boldsymbol{D}\boldsymbol{L}) \tag{14}$$

By approximating the system as LQ and recomputing the coupled DAREs for both Equation (13) and Equation (14), we converge to feedback policies that form the local Nash Equilibrium for zero-sum LQ games. Expanding these matrices provides our initial guess of the optimal action trajectory $\xi(x^0) = (u^{0:T}, w^{0:T})$. As a reminder, we do not use this trajectory for closed-loop control. Instead, we use this trajectory as a seed for the stochastic search presented in Section 4.2.

The traditional formulation of linear-quadratic approximations for zero-sum games can violate the eigenvalue conditions of Equation (11), even when a closed-form NE exists for the underlying non-LQ system. In practice, this can be alleviated by adjusting R_u and R_w manually. The gain matrices can be solved directly through Equation (11) or with readily available open-source software, such as Scipy's `solve_discrete_are` [47].

4.2 Refining the Action Trajectory

The initial action trajectory produced by the local equilibrium policies in Section 4.1 are optimal if the system is linear-quadratic. However, as established in Section 3, the environments that we are interested in may feature non-linear dynamics and non-quadratic costs, meaning that the LQ solution is merely a local approximation. These nonlinearities and nonconvexities cause the information gap $\overline{V}(x) - \underline{V}(x) > 0$. To be robust to the human, we must bridge the gap between this local optimum and the true upper value of the game $\overline{V}(x)$.

To tractably solve this nested optimization, we propose using the LQ solution as a highly-informed "warm start" that is subsequently refined through stochastic gradient descent. Specifically, we apply a nested Metropolis-Hastings (MH) sampling algorithm. While other Monte Carlo methods could be substituted within our general framework, the ergodic properties of the MH sampler are particularly well suited for escaping the local optima that frequently trap pure LQ-approximate methods [5].

As a reminder, our ultimate goal is to reach the robust robot policy defined in Equation (7), which can be treated as a nested optimization problem. In the outer loop the robot proposes a sequence of actions $u^{0:T}$ to minimize its cost, and in the inner loop the antagonistic human responds with actions $w^{0:T}$ that maximize that cost. Because the robust policy takes the form of a minmax optimization, our MH sampler is explicitly divided into two coupled phases [21]: an inner loop

Algorithm 1 Monte Carlo Linear-Quadratic Games (MCLQ). The LQ Approximation procedure is only called for $t = 0$. For $t > 0$, the Monte Carlo Search is seeded with the previous solution.

Require: f, J, β, M, N ▷ Dynamics and Cost Functions

procedure LQ Approximation(x, f, J)
  $\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{D} \leftarrow$ Linearize(f)
  $\boldsymbol{Q}, \boldsymbol{R}_u, \boldsymbol{R}_w \leftarrow$ Quadraticize(J)
  $\boldsymbol{K}, \boldsymbol{L} \leftarrow$ DARE Solution
  $\xi_u \leftarrow -\boldsymbol{K}\boldsymbol{x}$
  $\xi_w \leftarrow -\boldsymbol{L}\boldsymbol{x}$
  return ξ_u, ξ_w
end procedure ▷ Provides initial guess of ξ_u, ξ_w
procedure Monte Carlo Search(x, ξ_u, ξ_w, f, J)
  $\Delta_u \leftarrow 0, \Delta_w \leftarrow 0$
  $J_0 \leftarrow J(x, \xi_u, \xi_w)$
  for $m = 1 \dots M$ **do** ▷ Outer Loop: find the best-case robot action
    for $n = 1 \dots N$ **do** ▷ Inner Loop: find the worst-case human response
      $\Delta_n \leftarrow$ Perturb(ξ_w, Δ_w) ▷ Optionally use the safety margin λ or model $\hat{\pi}_H$
      $J_w \leftarrow J(x, \xi_u + \Delta_u, \xi_w + \Delta_n)$
      if $J_w > J_0 \vee \exp(\beta(J_w - J_0)) > \eta,\ \eta \sim U[0, 1]$ **then**
        $J_0 \leftarrow J_w$
        $\Delta_w \leftarrow \Delta_n$
      end if ▷ Updates worst-case human actions
    end for
    $\Delta_m \leftarrow$ Perturb(ξ_u, Δ_u)
    $J_u \leftarrow J(x, \xi_u + \Delta_m, \xi_w + \Delta_w)$
    if $J_u < J_0 \vee \exp(\beta(J_0 - J_u)) > \eta,\ \eta \sim U[0, 1]$ **then**
      $J_0 \leftarrow J_u$
      $\Delta_u \leftarrow \Delta_m$
    end if ▷ Updates robot response to human
  end for
  return $\xi'_u = \xi_u + \Delta_u, \xi'_w = \xi_w + \Delta_w$
end procedure

that models the worst-case human response, and an outer loop that optimizes the robot's robust commitment. An outline of our resulting approach is shown in Algorithm 1.

Inner Loop. The inner loop seeks to identify worst-case human actions that increase the robot's cost. Given a fixed robot commitment ξ_u, the adversarial human wants to maximize J. We define the target distribution for the human's trajectory as a Gibbs measure:

$$P_{\pi_{\mathcal{H}}}(\xi_w \mid \xi_u, x) \propto \exp\left(\beta J(x, \xi_u, \xi_w)\right) \tag{15}$$

where $\beta > 0$ is the inverse temperature. As $\beta \to \infty$, the exponential function heavily exaggerates differences in J and the probability measure collapses into a Dirac delta function centered strictly on the global maximum of J. Therefore, sampling from $\pi_{\mathcal{H}}$ at a high β is theoretically equivalent to solving the inner sup problem. Sampling directly from this distribution is intractable for continuous and high-dimensional systems. Instead, we use the Metropolis-Hastings sampler [12] by proposing random perturbations $\Delta\xi_w$ of the human's action. These perturbations are accepted if they increase

the total cost, or if they satisfy the acceptance rate:

$$\exp\left(\beta \cdot \left(J\left(x, \xi_u, \xi_w + \Delta\xi_w\right) - J\left(x, \xi_u, \xi_w\right)\right)\right) > \eta, \quad \eta \sim U[0,1] \tag{16}$$

In practice, using a β that is too large leads to numerical instability and reduces exploration, so we use a $\beta < 20$ and opt to do the comparison logarithmically. The inner loop repeats N times before terminating with a new human action sequence $\xi'_w = \xi_w + \Delta\xi_w^N$. Sampling random perturbations $\Delta\xi_w$ according to this scheme will find human actions that are the worst-case for a given robot action ξ_u. The mechanism for proposing actions human actions can instead use the nominal human model $\hat{\pi}_\mathcal{H}$; this process is detailed in Section 4.3.

Outer Loop. The outer loop takes the updated choice of human actions ξ'_w, and seeks to find robot actions that are robust to the human. Similar to the inner loop, we define the target distribution for the robust robot trajectory as:

$$P_{\pi_\mathcal{R}}(\xi_u \mid x) \propto \exp\left(-\beta \mathop{\mathbb{E}}_{\xi_w \sim P_{\pi_\mathcal{H}}}\left[J(x, \xi_u, \xi_w)\right]\right) \tag{17}$$

Like Equation (15), directly sampling from this distribution is difficult. We follow the same tractable sampling scheme as before. The outer loop modifies ξ_u by proposing random perturbations $\Delta\xi_u$. These perturbations are accepted if they decrease total cost, or if they satisfy the acceptance rate:

$$\exp\left(\beta \cdot \left(J\left(x, \xi_u, \xi'_w\right) - J\left(x, \xi_u + \Delta\xi_u, \xi'_w\right)\right)\right) > \eta, \quad \eta \sim U[0,1] \tag{18}$$

Importantly, the inner loop (i.e., the human) is *maximizing* the cost, while here the outer loop (i.e., the robot) is *minimizing* that same cost. This outer loop terminates after M iterations, and outputs a final robot action trajectory $\xi'_u = \xi_u + \Delta\xi_u^M$.

4.3 Balancing Performance and Safety

One key advantage of our approach is that — as we will show — it can be implemented in real-time. But another core aspect is that the designer can tune the *safety margin*, i.e., how conservative the robot's behavior is. Recall that $\hat{\pi}_\mathcal{H}$ is a predictive model of the human's actions. We recognize that the real human will inevitably deviate from $\hat{\pi}_\mathcal{H}$; but how different should we expect the human's actions to be? Within a zero-sum game formulation, if the human can take any action at any time, then the robot is forced to overly conservative behaviors (e.g., the autonomous drone always moving away from the human). Unlike LQ approximations, our approach is structured to resolve this conflict between performance and safety. Specifically, during Monte Carlo sampling we can impose hard constraints on the perturbations $\Delta\xi_w$ so that they are close to the predictions of our human model. Let $\hat{\xi}_w \sim \hat{\pi}_\mathcal{H}$ be a predicted human trajectory. We can limit the range of considered human actions such that:

$$\|\hat{\xi}_w - (\xi_w + \Delta\xi_w)\|_2^2 \leq \lambda \tag{19}$$

where λ is a design parameter that determines conservatism. As $\lambda \to \infty$, the search space approaches the original trajectory space $\Xi_\mathcal{H}$, and as $\lambda \to 0$, the robot is increasingly confident in its human model. Importantly, Equation (19) does not determine *if* the human's behavior aligns with our nominal model: it restricts the search space to trajectories that are *similar* to our nominal model.

Instead of restricting the sampled actions such that they are close to the nominal model's prediction, we can change the sampling distribution of Equation (15) entirely. If we want the adversarial inner-loop to align with our expectation of the human's behavior, we can constrain the

Gibbs measure of Equation (15) to be:

$$\arg\max_{P \in \mathcal{P}_{\pi_{\mathcal{H}}}} D(P \parallel \hat{\pi}_{\mathcal{H}}), \quad \text{s.t. } D(P \parallel \hat{\pi}_{\mathcal{H}}) \leq \lambda \tag{20}$$

where D is a divergence metric (such as the Kullback-Leibler Divergence [30]). This accomplishes the same goal as Equation (19): limiting conservatism in the robot policy. However, instead of enforcing the restriction λ on the sampled trajectories, we instead enforce it on the action distribution itself. In practice, this is accomplished by changing the perturbation distribution $\Delta\xi_w \sim P$ from a uniform distribution to one that aligns more closely — according to the conservatism parameter λ — with the nominal human model.

4.4 Theoretical Analysis

Taken together, the LQ game in Section 4.1 provides a first guess at the robot's action trajectory ξ. This action trajectory is then refined by nested Metropolis-Hastings samplers in Section 4.2 that update ξ to solve for the robust robot policy defined in Equation (7). To limit conservatism, the inner loop of the nested Metropolis-Hastings samplers can be tuned with a design parameter λ and a nominal human model using Section 4.3. The resulting trajectory ξ' contains the sequence of actions a robust robot should execute over the next T timesteps; this robust action sequence is recomputed online (i.e., at each timestep t). We compute the LQ game approximation only once, at the beginning of the interaction. At every later timestep we warm-start the nested Metropolis-Hastings samplers from the previous solution: after the robot executes the first action of its planned trajectory, we shift the remaining actions forward by one timestep and repeat the final action to fill the horizon. This seed has already been refined on the true dynamics and cost, and because consecutive problems differ by only one timestep, it starts the search close to the new robust trajectory. Our analysis below does not rely on the quality of this seed: the convergence rates hold from any starting trajectory (Appendix A.2). The seed does matter in practice; early stopping can be employed as needed, but our analysis assumes a fixed number of inner and outer iterations for the MCLQ algorithm.

MCLQ is applied in a receding-horizon fashion. At each timestep the robot solves the open-loop problem in Equation (7) over the next T steps, executes only the first action, and then solves the problem again from the newly observed state. We search over open-loop action sequences because they are fast to evaluate, which allows the robot to replan at every timestep. The analysis below applies to each of these open-loop problems; we do not provide guarantees on the closed-loop behavior that results from replanning (see Section 7). In what follows, we will first show theoretically that this approach converges towards the robust policy even when LQ approximations are inaccurate, then experimentally validate this claim in Sections 5 and 6.

Convergence Analysis. So far we have described how MCLQ searches for robust robot behavior. Here we ask a natural follow-up question: how close is the trajectory that MCLQ returns to the robust policy in Equation (7)? Because MCLQ *samples* trajectories rather than solving for them directly, the answer is expressed probabilistically. Specifically, we bound the probability that the robot's planned actions ξ_u' lie within some distance r of the robust actions:

$$\xi_u^\star = \arg\min_{\xi_u} F(\xi_u), \qquad F(\xi_u) = \sup_{\xi_w} J(x, \xi_u, \xi_w) \tag{21}$$

where F is the robot's cost under the worst-case human response. Our analysis relies on the assumptions from Section 3 — the dynamics and costs are twice continuously differentiable and the action sets $\mathcal{U}$ and $\mathcal{W}$ are compact and convex — and on one property of the algorithm: the perturbations proposed by each loop can reach any trajectory in a finite number of steps. There are three separate reasons why the trajectory chosen by MCLQ may not match the optimal trajectory $\xi_u^\star$. We outline these three below and report the convergence rate of MCLQ to robust behavior.

Finite β. Suppose for now that the outer loop evaluates F exactly (we remove this simplification under *Finite N* below). Then the outer loop is a Metropolis-Hastings sampler whose target is the Gibbs distribution $P_\beta(\xi_u) \propto \exp(-\beta F(\xi_u))$, i.e., Equation (17) with the inner expectation replaced by the exact worst case. This target is a distribution: our sampler will sometimes return trajectories away from $\xi_u^\star$. As β increases, this distribution concentrates on the minimizers of F [25]: here we quantify how quickly that occurs. To measure how often this occurs, let $\Delta(r)$ be the *optimality gap* at radius r:

$$\Delta(r) = \inf_{\|\xi_u - \xi_u^\star\| > r} \left[F(\xi_u) - F(\xi_u^\star)\right] \tag{22}$$

Put another way, every trajectory further than r from the optimum is worse by at least $\Delta(r)$. If several robot trajectories are equally robust, $\Delta(r)$ is zero until r exceeds the distance between them. Each of these trajectories is therefore at most $\exp(-\beta\Delta(r))$ times as likely as the optimum, and so:

$$P_\beta\big(\|\xi_u - \xi_u^\star\| > r\big) \leq C_1 \exp\left(-\beta\Delta(r)\right) \tag{23}$$

where C_1 depends on the size of the action space and grows at most polynomially in β (see Appendix A.1).

Finite M. After M outer iterations the sampler may not yet have reached P_β. Because the action space is compact and the proposals can reach any trajectory, the outer chain is uniformly ergodic [12, 46] (see Theorem 8 of [39] and Appendix A.2). An implication of this ergodicity is that the total variation distance between the distributions of ξ_u' and P_β decays as $C_2 \exp(-cM)$ for some constants $C_2, c > 0$. We emphasize that this rate depends solely on the number of outer iterations.

Finite N. In practice the outer loop does not evaluate F exactly. On line 22 of Algorithm 1, each robot proposal is scored against the human trajectory currently held by the inner loop, rather than against a worst-case response. The outer loop is therefore a *noisy* Metropolis-Hastings sampler [1]: occasionally, the inner loop's approximation causes the outer loop to accept a proposal that it should have rejected, or to reject one it should have accepted. We let $\varepsilon_{N,\beta}$ denote the probability that this happens at any point during the search. This error cannot be found by applying single-loop results to each sampler separately. The inner loop is warm-started from wherever it stopped at the previous outer iteration, so the human's trajectory depends on the entire history of the search, and the two samplers are not independent. We account for this sequential dependency in Appendix A.3: because the inner loop's convergence rate holds from *any* starting trajectory, our bound holds no matter what happened earlier in the search. In Appendix A.3 we derive $\varepsilon_{N,\beta}$ explicitly and show that it has three intuitive properties:

- it is proportional to how strongly the robot's actions change the human's incentives, and is exactly zero when the human's worst-case response does not depend on the robot;
- it is proportional to the size of the outer loop's perturbations;
- it decreases exponentially in the number of inner iterations N, down to a value that depends on how clearly the human's worst-case response stands out from alternatives.

Overall Bound. Combining these three sources of error, the probability that MCLQ returns a robot trajectory within r of the robust trajectory is at least:

$$P\big(\|\xi_u' - \xi_u^\star\| \leq r\big) \geq 1 - C_1 \exp\left(-\beta\Delta(r)\right) - C_2 \exp\left(-cM\right) - \varepsilon_{N,\beta} \tag{24}$$

Equation (24) shows how each of the designer's parameters affects MCLQ. The inverse temperature β determines how sharply the sampler concentrates on good trajectories; the number of outer iterations M determines whether the sampler has reached that concentrated distribution; and the number of inner iterations N determines how accurately the outer loop accounts for the worst-case human. These parameters also interact: increasing β tightens the first term, but it slows the mixing of both loops (see Appendix A.2) and can increase the nested error $\varepsilon_{N,\beta}$.

Simplifications. When the game is linear-quadratic we can be more precise. In this case the worst-case cost F is itself quadratic, with curvature of at least $\mu > 0$ in every direction. Ignoring the action bounds, the target P_β is then a Gaussian centered on the robust trajectory, and each element of the trajectory has standard deviation at most $1/\sqrt{\beta\mu}$ (see Appendix A.1):

$$P\left(|\xi'_{u,i} - \xi^\star_{u,i}| \le r\right) \ge 1 - 2\exp\left(-\beta\mu\frac{r^2}{2}\right) - C_2\exp\left(-cM\right) - \varepsilon_{N,\beta} \tag{25}$$

Equation (25) states how close *each element* of the trajectory is to the robust trajectory. The full trajectory contains $d = mT$ elements (an m-dimensional action at each of T timesteps), and their errors accumulate: the distance between the robot's trajectory and the robust trajectory is typically around $\sqrt{d/(\beta\mu)}$ rather than 0. This is the width of the target distribution, and so running the sampler for longer cannot bring the full trajectory any closer; only increasing β can. Fortunately, MCLQ is applied in receding horizon, and the robot only ever executes the *first* action of each trajectory, which has just m elements.

Comparison to LQ Approximations. We now consider three cases for the underlying environment dynamics.

- **LQ Systems**: For a linear-quadratic system the DARE presented in Equation (11) finds the equilibrium between the two agents, if it exists. The initial trajectory is therefore the robust robot policy, and Monte Carlo sampling cannot improve upon this initial guess without restricting λ.
- **Non-Linear Systems with Quadratic Cost:** If the system is non-linear but has quadratic cost, the LQ approximation is only accurate near the linearization point, and MCLQ refines the initial trajectory towards $\xi^\star_u$ according to Equation (24).
- **Non-Linear Non-Quadratic Systems:** If the system is neither linear nor quadratic, the error in the LQ approximation can be large. MCLQ still converges towards $\xi^\star_u$ according to Equation (24) with a rate determined by the optimality gap $\Delta(r)$ of the underlying game.

5 SIMULATIONS

In theory, our proposed MCLQ method moves towards robust, game-theoretic behaviors while offering real-time performance. Here we test our theoretical claims by comparing our approach to exact Hamilton-Jacobi methods and tractable LQ approximations. More specifically, we perform controlled experiments where simulated agents interact in three environments: point-mass, driving, and manipulator (see Figure 2). Each environment contains two agents. The simulated robot is attempting to complete its task (e.g., reaching a goal) while simulated humans move in its proximity. The robot must avoid collisions with these simulated humans — even when the humans take unexpected or noisy actions.

Independent Variables. We vary the robot's controller across four levels. To test LQ approximations, we include **DARE-nA** [52] and **ILQGames** [20] approaches. These methods iteratively approximate the dynamics and cost as a LQ system and solve for the resulting Nash Equilibrium. We used the official repositories for **DARE-nA** and **ILQGames** when applicable. To test exact solutions, we next solve for the true Nash Equilibrium (**NE**) using the Bellman Equation. Finally, we evaluate our **MCLQ** approach from Section 4.

Environments. Below we describe the three simulated environments. All simulations were performed on an AMD Ryzen 7000 Series 5 CPU with multithreading enabled.

Point-Mass. Here the simulated human and robot move in a 2D plane. The initial positions and velocities of both agents are randomized; across an interaction of $T = 30$ timesteps, the robot attempts to reach a fixed goal location. Both agents have linear dynamics such that $x^{t+1} = x^t + u^t + w^t$,

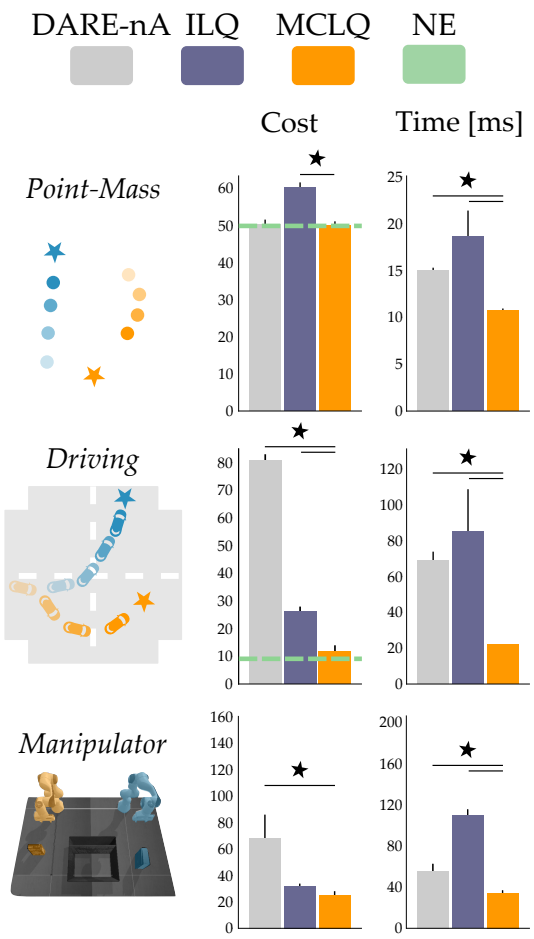


Fig. 2. Simulation results across *point-mass*, *driving*, and *manipulator* environments. (Left) We plot the cost and (Right) computation time averaged over 100 simulations. Computation time is the number of milliseconds per robot action (normalized by the number of timesteps per trajectory). MCLQ computes the LQ approximation once at $t = 0$, where it accounts for approximately 7% of the computation time; across the entire interaction it accounts for less than 0.3% of computation time. In practice the ratio between the LQ approximation time and the MH time can be tuned by adjusting the number of iterations of the MH refinement. In non-LQ settings the computation time for NE is prohibitively high; e.g., in *driving* the NE computation time exceeded one hour. We did not calculate NE in the 26-dimensional *manipulator* environment. Error bars show standard deviation and an $*$ denotes statistical significance.

and the robot's cost is quadratic (considering both distance to goal and distance to human). Because this environment is a linear-quadratic game, we anticipate that LQ approximations should find the Nash Equilibrium.

Driving. Our driving environment is taken from the iLQ baseline [20]. As before, one simulated human and robot move in a 2D plane with randomized initial configurations. The dynamics of each agent follow a nonlinear bicycle model. Here the robot's cost function is not quadratic:

$$j(x, u, w) = \|x_{\mathcal{R}}^t - g\|^2 + \eta \exp\left(\frac{\|x_{\mathcal{R}}^t - x_{\mathcal{H}}^t\|^2}{-\eta}\right) + u' R_u u$$

where g is the goal position the robot is trying to reach, $x_{\mathcal{R}}$ is the robot's position, $x_{\mathcal{H}}$ is the human's location, and η scales the cost of approaching the human. Each interaction lasts a total of $T = 30$ timesteps.

Manipulator. Our final environment contains two 7-DoF robot arms in PyBullet. We treat one of these arms as the simulated human, and the other as the robot. Both agents are trying to pick up and place objects within a shared workspace. The dynamics of the arms and the task are nonlinear, and the system state $x \in \mathcal{X}$ is 26-dimensional (including manipulators and objects). We leverage a cost function similar to *driving*. The robot moves to reach, grasp, and place items while avoiding collisions with the other arm. We separate this task into two separate sub-tasks: *reaching* and *placing*, each with separate cost functions. Settings with grasping violate the smooth dynamics assumptions of Section 3. To still use LQ approximations for the environment, the sub-task is automatically switched when the *reaching* sub-task's cost is below a certain threshold.

Theoretical Assumptions. Our analysis in Section 4.4 relies on the assumptions of Section 3: the dynamics and costs are smooth, and the action sets are compact and convex. Both assumptions hold in the *point-mass* and *driving* environments, with the separation of the high-level task of *manipulator* into two separate sub-tasks satisfying these assumptions as well.

Simulated Human. We simulate humans as bounded-rational agents that noisily optimize their cost function J_H:

$$\begin{aligned} \pi_{\mathcal{H}}\left(w^t \mid x^t\right) &\propto \exp\left(-\alpha \cdot J_H(x^t, w_i^t)\right) \\ \text{s.t. } x^{t+1} &= f(x^t, u^{t_0}, w^t) \end{aligned} \tag{26}$$

Increasing $\alpha \to \infty$ causes the human to always take the optimal action, while decreasing $\alpha \to 0$ causes the human to act randomly [19]. In our simulations we set $\alpha = 7.5$. When computing the cost for future timesteps the simulated human assumes the robot will repeat its most recent action u^{t_0} (e.g., the robot will keep moving with the same velocity). In each environment the human had a task that was independent of the robot's objective — for instance, in *driving* the human tried to reach their own goal position.

Results. The results from our first simulation are summarized in Figure 2. Across all three environments, MCLQ achieved costs that were closest to the upper value of the game (**NE**) while also requiring the least amount of computation time. In *point-mass* we highlight that all methods reached similar cost; this matches our expectations, because *point-mass* was an LQ system. By contrast, in non-LQ systems (*driving* and *manipulator*) the LQ approximations made by DARE-nA and ILQ fell short, leading to suboptimal performance. Computing the exact solution with NE was time-consuming and not always feasible: indeed, in the 26-dimension *manipulator* task, we were unable to compute the true NE because of the high-dimensional and continuous state-action space.

Adjusting Safety Margins. As discussed in Section 4.3, one feature of our approach is the designer-selected safety margin. MCLQ robots reason over the worst-case human action within bounds λ. By increasing λ in Equation (19) the designer makes the robot more risk-averse (i.e., the robot considers larger deviations from the nominal human model). Conversely, decreasing λ makes the robot more risk-seeking (i.e., the robot increasingly relies on its human model). Within our simulations from Figure 2 we left this value of λ fixed at risk-neutral behavior. Now we explore how increasing and decreasing λ changes the robot's performance in a modified *point-mass* environment.

Our extended *point-mass* environment includes one to ten humans that are navigating to goal positions. The robot seeks to reach to its own goal while avoiding these randomly generated agents. To find an initial plan, the robot is equipped with a nominal human model — it assumes each human will move in a straight line towards their goal. In practice, however, our simulated humans deviate from this model when nosily optimizing their cost function. Figure 3 plots the robot's performance

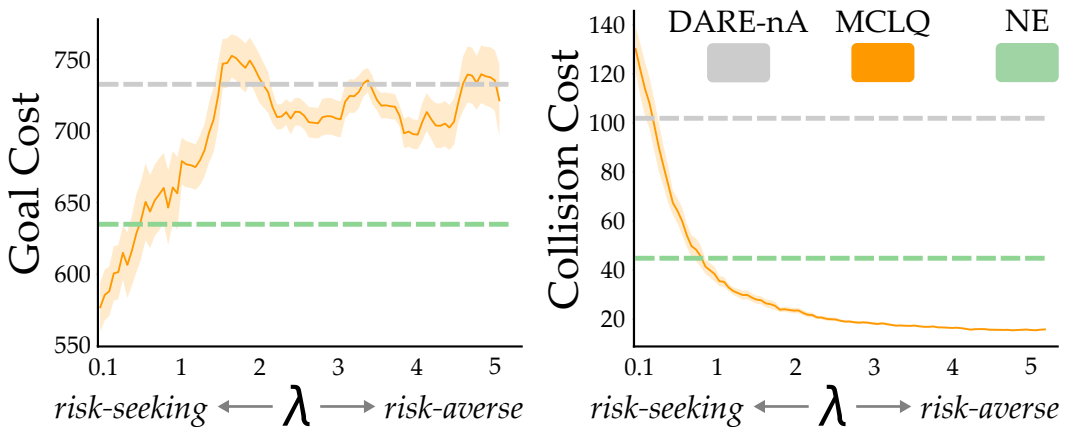


Fig. 3. Simulation results for a modified point-mass environment where we adjust the safety margin λ in MCLQ. Increasing λ causes the MCLQ robot to consider a wider range of worst case human actions, resulting in more conservative behavior. Conversely, decreasing λ causes the MCLQ robot to increasingly rely on its nominal human model. Unlike LQ approximations, our proposed method gives designers the flexibility to tune λ and adjust the safety margin. In practice, this results in risk-averse robots that avoid all possible collisions but reach the goal more slowly, or risk-seeking robots that allow some potential collisions and rapidly move to the goal.

as a function of λ. When λ increases the robot becomes more risk-averse: the MH sampler accounts for a wider range of adversarial human actions. This causes the robot agent to stay farther from humans (reducing collisions), but also leads to longer paths (increasing distance). Conversely, lower values of λ cause to risk-seeking behavior where the robot relies on its human model. If humans stick to this model, the robot avoids collisions while minimizing distance. But when humans deviate, the number of collisions increase. Overall, this simulation supports our theoretical description of λ and demonstrates how designers can leverage MCLQ to tune the safety margin.

We further test how designers can change the human model within Equations (15) and (16) by comparing the performance of MCLQ across different human models. We leverage the noisily-optimal human model from Equation (26) with both varying levels of rationality $\alpha \in \{2.5, 10.0, 20.0\}$ and cost functions $J_H \in \{J_H^1, J_H^2, J_H^3\}$:

$$J_H^1(x, u, w) = \|x_H^N - g_H\|_{P_H}^2 + \sum_{t=0}^{N-1} \|x_H^t - g_H\|_{Q_H}^2 + \|w^t\|_{R_H}^2 \tag{27}$$

$$J_H^2(x, u, w) = \|x_H^N - g_H\|_{P_H}^2 + \sum_{t=0}^{N-1} \|x_H^t - g_H\|_{Q_H}^2 + \eta \exp\left(\frac{\|x_R^t - x_H^t\|_2^2}{-\eta}\right) + \|w^t\|_{R_H}^2 \tag{28}$$

$$J_H^3(x, u, w) = \|x_H^N - g_H\|_{P_H}^2 + \sum_{t=0}^{N-1} \|x_H^t - x_R^t\|_{Q_H}^2 + \|w^t\|_{R_H}^2 \tag{29}$$

$$\text{s.t.} \quad x^{t+1} = f(x^t, u^t, w^t), \quad u^t \sim \pi_{\mathcal{R}}(x^t) \tag{30}$$

where Q_H, R_H, and P_H are fixed, time-invariant matrices and $\pi_{\mathcal{R}}$ is our proposed method for selecting robust robot actions. These cost functions cause the human to move towards the goal while minimizing their action size, but they also ensure that the human follows one of three general behaviors: ignore the robot's position (27), avoid the robot's position (28), and follow the robot

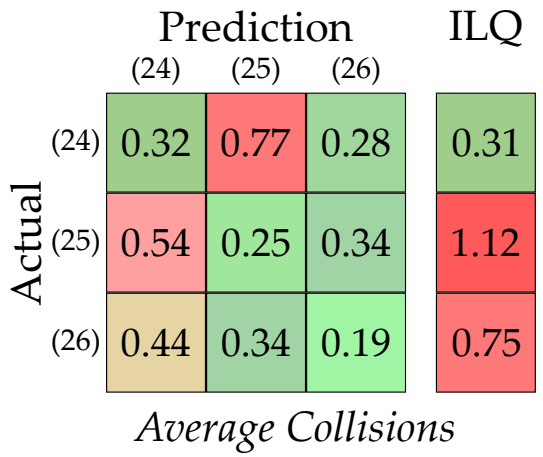


Fig. 4. Generally, when the human model aligns with the actual human behavior, MCLQ avoids worst-case scenarios (here, collisions). The annotations (24), (25), and (26) correspond to the human models in Equations (27-29). Here we define a collision as an instance where the human is within 5 units of the robot. The results are averaged across 1089 trials with 9 variations of human models in the point-mass environment. Note that values along the diagonal tend to be the lowest, indicating that when the actual and modeled motions of the human model align, the robot maintains safety. Overall, when a pessimistic human model is used (such as Equation (29)), the robot tends to avoid the human more frequently. In this experiment, we define a "collision" as a state where the human and robot agents are within 5 units of each other. The initial configuration of the agents is randomly sampled from $U[-100, 100]$. Similar to *Point Mass*, each agent has two degrees of freedom (i.e., $\mathcal{X} \subset \mathbb{R}^4$ and $\mathcal{U} = \mathcal{W} \subset \mathbb{R}^2$) and each interaction lasts for $T = 100$ timesteps.

(29). Intuitively, each of these cost functions could be a reasonable approach for predicting the human's behavior, and designers may choose any of these (or other similar functions). Our goal is to demonstrate that even when the human model is incorrect — and the human follows another underlying behavior pattern — our approach to robust interact still provides a safety measure.

As expected, when the actual human aligns with the behavior that MCLQ is modeling, the number of average collisions decreases (Figure 4). Notably, when the predicted human model is more pessimistic (such as in Equation (29)), MCLQ tends to avoid the human more at the expense of goal convergence. We test MCLQ with a safety margin $\lambda = 1.0$ according to Equation (19) and a non-quadratic cost function similar to the *Driving* environment. Notably, the iterative LQ approximation baseline has similar performance for Equation (27), but fails for complex behaviors that are more difficult to approximate.

6 USER STUDY

Our simulations from Section 5 support our theoretical analysis, and suggest that MCLQ converges towards maximally-robust behaviors while minimizing computation time. We next evaluate our method in a real-world setting with $N = 24$ in-person human participants. Here users walk around a room to complete an assembly task while an autonomous drone circles that room to inspect the parts (see Figure 1). The robot modifies its high-level trajectory to avoid getting to close to the human workers. We compare two *real-time* methods for safe interactions that adjust the drone's behavior: ILQ [20] and MCLQ. We selected ILQ here because it was the best performing real-time baseline from our simulations.

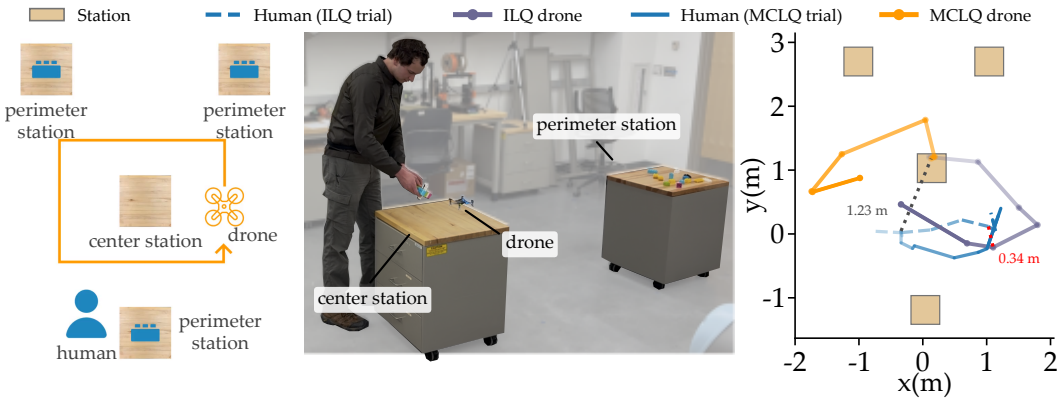


Fig. 5. An overview of our real-world experimental setup. (Left) Top view of the workspace. Participants built a tower at the center station using blocks from the three perimeter stations, while the drone circled the center station to monitor the workspace. (Middle) A side view of the same layout. Here the human is stacking the tower at the center station while the drone is taking an evasive maneuver. (Right) A sample interaction from a participant in our user study. We selected a 10 second window from both MCLQ and ILQ in which the participant walked nearly the same path (dashed and solid blue lines) and the drone started from the same position (the center station). The ILQ drone crossed the participant's path and came within 0.34 m of them, while the MCLQ drone moved across the side of the center station and stayed at least 1.23 m away. Positions were recorded approximately once per second.

Experimental Setup. Participants interacted with a Crazyflie 2.1+ (Bitcraze) during the assembly task. The participant's objective was to construct a Lego tower at the central station. The blocks needed for building that tower were scattered in three other stations located along the perimeter of the workspace. Accordingly, users needed to move back and forth through the workspace to acquire blocks and build their tower. The drone's objective was to monitor the workspace during the task by completing as many revolutions around the central station as possible. As the drone moved around the central station it repeatedly intersected with the human worker: here the drone should take actions that avoid the moving participant. Participants interacted with the drone separately (i.e., the drone interacted with one user at a time). Each participant wore a mobile VIVE tracker around their waist. Three base stations were mounted on tripods outside of the play area that tracked the participants position using Steam VR Tracking Version 2.0. We ensured that the VIVE trackers were properly calibrated before each user interacted with the drone. An overview of the experimental setup is shown in Figure 5

Participants and Procedure. We recruited 24 participants (6 female, age 24.5 ± 4.5) from our university community for this user study. Of the 24 participants, 8 had not used drones and 5 did not have experience with robotics. Participants received monetary compensation for their time and provided informed written consent according to university guidelines (IRB #23-1237).

We leveraged a within-subjects design where every participant interacted with ILQ for five minutes and MCLQ for five minutes. To prevent the participants from always going back and forth between the same stations, we instructed users to move according to three movement patterns: clockwise, counter-clockwise, and randomly. Both the order of the methods and the movement patterns were counterbalanced (i.e., half of the participants started with MCLQ, and half with ILQ). Participants were never told which algorithm the drone was using.

We did not conduct an *a priori* power analysis. Instead, we performed a sensitivity power analysis to determine the smallest effect our sample could reliably detect. We computed this for our primary

test, a two-tailed Wilcoxon signed-rank test for matched pairs with $\alpha = 0.05$, power $1 - \beta = 0.80$, and $N = 24$ participants. We used the noncentral T-distribution with the asymptotic relative efficiency correction for the Wilcoxon test (normal parent distribution), the method implemented in G*Power [17], and verified the result with a Monte Carlo simulation of 20000 studies. The minimum detectable effect is $d_z = 0.61$ (0.71 for 90% power), where d_z is the mean paired difference divided by the standard deviation of the paired differences. Our sample can therefore reliably detect medium-to-large within-subjects effects, but it has limited power for smaller effects.

Human Model. We model the participants behavior as noisily-optimal using a boltzmann-rational human model, like in Equation (26). Here the human model has two separate objective functions: one that is active when the human picks up a block from a station on the perimeter of the workspace, and another that is active after a human places a block on the center station. We model the human as noisily-optimal with point-mass dynamics:

$$\pi_{\mathcal{H}}(w \mid x, J) \propto \exp\left(-\beta_{\mathcal{H}} \cdot J(x, w)\right), \quad \text{s.t. } f(x, w) = x + w \tag{31}$$

where J changes based on the two conditions mentioned earlier. First, when the human has picked up a block from one of the external stations:

$$J_1(x, w) = \|x - x_{\text{central station}}\|^2 \tag{32}$$

and secondly, when the human has placed a block at the central station:

$$J_2(x, w) = \min_{x_{\text{ext}}} \|x - x_{\text{ext}}\|^2 \tag{33}$$

We do not directly determine when the human has picked or placed a block. Instead, we use J_1 after the human has entered a radius of one of the exterior stations, and use J_2 after the human has entered a radius of the central station.

Dependent Measures — Objective. We using tracking devices (VIVE) to measure the states and actions of the drone and human at each timestep of the interaction. To assess objective performance, we considered two metrics. For our first metric (*collisions*) we counted the number of times the drone was within a radius of 0.5m from the human. Our second metric (*revolutions*) is the number of revolutions the drone completed within the five minute trial. Lower values for *collisions* indicate that the drone is maintaining safety, while higher values for *revolutions* show that the drone is performing the task more efficiently.

Dependent Measures — Subjective. After interacting with each control algorithm, participants completed a 7-point Likert scale survey. This survey assessed the user's subjective preferences along three multi-item scales. We asked users:

(1) If they felt *safe* during the interaction,
(2) If they thought the drone was *attentive* to their position,
(3) If they thought the drone's movements were *predictable*.

Hypotheses. We had two hypotheses for this user study:

> **H1. MCLQ** *will modify the drone's actions to reduce the number of collisions and improve subjective feelings of safety.*
>
> **H2. MCLQ** *will complete a similar number of revolutions as the iLQ baseline.*

Data Analysis. Because each participant experienced both algorithms, we compared conditions with paired tests. Occasionally, the VIVE tracker would lose connection with the base station and we would need to restart the tracker program. This causes the interaction to be recorded in multiple segments; we combine these segments so that each participant contributes one value per measure and condition. We used Wilcoxon signed-rank tests for all measures and report the matched-pairs

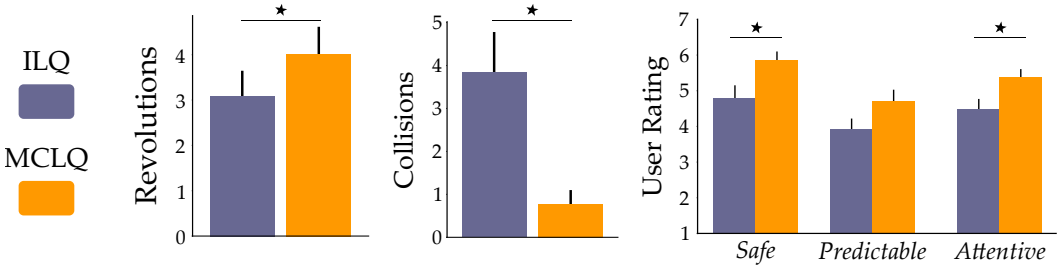


Fig. 6. Results from our user study in Section 6. Participants walked around a room to assemble a tower; a drone completed revolutions around the same workspace to monitor the human's progress (also see Figure 1). (Left) The average number of revolutions the drone completed and the average number of collisions. Here "collisions" occurred when the drone was within 0.5 meters of the human. The proposed MCLQ algorithm adjusts the robot's behavior to increase safety (fewer collisions) while also enhancing performance (more revolutions). (Right) After interacting with each algorithm participants answered survey questions about how safe, predictable, and attentive the robot was. Ratings suggest that participants perceived MCLQ to be a safer system. Error bars show standard deviation and an $*$ denotes statistical significance ($p < .05$).

rank-biserial correlation r_{rb} as the effect size ($|r_{rb}| \approx 0.1, 0.3$, and 0.5 indicate small, medium, and large effects respectively). Positive values of r_{rb} indicates higher values with MCLQ.

Results. The results from our in-person user study are summarized in Figure 6. To assess **H2**, we measured the number of revolutions that each method completed per interaction: a higher number indicates more efficient performance. The MCLQ drone completed more revolutions than ILQ ($M = 7.38, SD = 1.13$ vs. $M = 6.00, SD = 1.41$) and a Wilcoxon signed-rank test showed that this result was significant ($W = 0, p < 0.001, r_{rb} = 1.00$). Of the 24 participants, 19 saw more revolutions with MCLQ, five saw the same amount, and none saw fewer. This suggests that the safety improvements made by MCLQ are not coming at the expense of overly conservative behavior: the MCLQ drone is still performing its high-level monitoring task to an efficient degree.

Given that MCLQ is not sacrificing task performance, the key question becomes the *safety* of the robot's behavior. We analyzed **H1** along two levels: objective safety (maintaining a minimum distance between agents) and subjective safety (the user's perception of the system). For objective safety, MCLQ led to fewer collisions than ILQ ($M = 1.42, SD = 1.98$ vs. $M = 7.21, SD = 5.67$), with a Wilcoxon signed-rank test showing that this result is significant ($W = 0, p < 0.001, r_{rb} = -1.00$). Of the 24 participants, 22 experienced fewer collisions with MCLQ and none experienced more than with ILQ. The participants' responses to our Likert-scale survey align with these objective results: users subjectively perceived the MCLQ robot to be significantly safer ($M = 5.81, SD = 1.21$ vs. $M = 4.75, SD = 1.79$; $W = 47, p = 0.010, r_{rb} = 0.63$) and more attentive ($M = 5.38, SD = 1.11$ vs. $M = 4.46, SD = 1.38$; $W = 52, p = 0.026, r_{rb} = 0.55$). Users also rated the MCLQ drone's actions as more predictable, although this difference was not statistically significant ($M = 4.69, SD = 1.52$ vs. $M = 3.88, SD = 1.39$; $W = 71.5, p = 0.073, r_{rb} = 0.43$). In their free response comments, participants stated that with the MCLQ drone they "*felt safer*" and that the drone was "*more reactive and predictable.*"

Discussion. Our user study results mirror our simulated experiments. In Section 5, LQ approximations fell short in environments with nonlinear dynamics and non-quadratic costs. The drone's task has a similar structure: its cost for approaching the human is not quadratic. Additionally, our nominal human model is simple, so participants often deviated from it. Under these conditions ILQ brought the drone close to the participant much more often than MCLQ (Figure 5). Importantly,

MCLQ's robustness did not make the drone overly conservative. Worst-case methods are often associated with conservative behavior [8, 45], but MCLQ is both robust to the human's behavior *and* more performant than ILQ. The participants' ratings align with these objective results, as they perceive the drone to be safer and more attentive, which is consistent with prior work on human-robot interaction. Ratings of predictability also favored MCLQ, but this difference was not significant. One possible explanation is that MCLQ replans with a stochastic search, so its avoidance maneuvers may be harder for people to anticipate than a fixed loop. Adding legibility objectives to the robot's cost is one direction that might make these robust behaviors easier to predict.

7 CONCLUSION

In this paper we presented a real-time safety approach for human-robot interaction. Our approach seeks robot behaviors that are robust to noisy and unexpected human behaviors within designer-specified bounds, where safety is encoded as a soft cost. To achieve this game-theoretic safety with tractable computation, we combined linear-quadratic approximations with stochastic local searches. Our theoretical and empirical analysis showed the resulting MCLQ algorithm converges towards robust robot policies while providing flexible and efficient implementation. Across multiple simulations and a user study, we observed that MCLQ advances both objective and subjective safety measures when compared to state-of-the-art control alternatives.

Limitations. MCLQ is a step towards control policies that are robust to human disturbances, but our approach has limitations. Since the stochastic search is performed on the underlying system directly, MCLQ is not restricted to systems that can be approximated as linear-quadratic. One key assumption in our work is that the objective function and state dynamics can be evaluated quickly. The inner loop of the double Metropolis-Hastings sampler is somewhat expensive: every perturbation of the original trajectory mandates recalculating the dynamics and cost. Performing this recalculation quickly is not always feasible, such as in fluid dynamics or with visuomotor policies. MCLQ plans open-loop. At each timestep the robot commits to a T-step plan and reasons about how the worst-case human would respond to that plan, even though the robot will replan at the next timestep. This makes the robot more conservative than necessary. Our analysis applies only within each planning problem, not to the closed-loop behavior across replanning steps. Because each problem looks only T steps ahead, the robot may also commit to behaviors whose consequences appear beyond the horizon. Increasing the horizon reduces this risk, but increases computation.

MCLQ is also only robust to human actions within the designer-specified bounds around its nominal human model. If the human rapidly changes their intent and acts outside these bounds, the robot does not consider those actions until its nominal model is updated. In addition, our analysis assumes smooth dynamics and costs (Section 3). Tasks with discrete transitions, such as the grasping step in our *manipulator* environment, violate this assumption, and so our analysis does not apply across these transitions; for such tasks, we require separate plans for each discrete transition.

Our user study recruited 24 participants from our university community. They were young, mostly experienced with robotics, and mostly male. Whether these results generalize to other populations remains untested. Our sensitivity analysis shows that the study had 80% power only for effects of $d_z \geq 0.61$. The objective effects were well above this threshold ($d_z = 1.24$ for Collisions and 1.10 for Revolutions), but the subjective effects were near or below it ($0.44 < d_z < 0.58$). The study may therefore have been underpowered to detect differences in perceived predictability.

Finally, our physical evaluation was limited to human-drone interaction; while our simulations include driving and manipulation, the performance of MCLQ with other physical platforms and real human partners remains to be tested.

REFERENCES

[1] Pierre Alquier, Nial Friel, Richard Everitt, and Aidan Boland. 2016. Noisy Monte Carlo: Convergence of Markov chains with approximate transition kernels. Statistics and Computing 26, 1 (2016), 29–47.

[2] Aaron D Ames, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada. 2019. Control barrier functions: Theory and applications. In 2019 18th European control conference (ECC). Ieee, 3420–3431.

[3] Aaron D Ames, Xiangru Xu, Jessy W Grizzle, and Paulo Tabuada. 2016. Control barrier function based quadratic programs for safety critical systems. IEEE transactions on automatic control 62, 8 (2016), 3861–3876.

[4] Tianjiao An, Xinye Zhu, Mingchao Zhu, Bing Ma, and Bo Dong. 2023. Fuzzy logic nonzero-sum game-based distributed approximated optimal control of modular robot manipulators with human-robot collaboration. Neurocomputing 543 (2023), 126276.

[5] Christophe Andrieu and Éric Moulines. 2006. On the ergodicity properties of some adaptive MCMC algorithms. The Annals of Applied Probability 16, 1 (2006), 1462–1505.

[6] Andrea Bajcsy, Somil Bansal, Eli Bronstein, Varun Tolani, and Claire J Tomlin. 2019. An efficient reachability-based framework for provably safe autonomous navigation in unknown environments. In IEEE Conference on Decision and Control. 1758–1765.

[7] Somil Bansal, Andrea Bajcsy, Ellis Ratner, Anca D Dragan, and Claire J Tomlin. 2020. A Hamilton-Jacobi reachability-based framework for predicting and analyzing human motion for safe planning. In IEEE International Conference on Robotics and Automation. 7149–7155.

[8] Somil Bansal, Mo Chen, Sylvia Herbert, and Claire J Tomlin. 2017. Hamilton-Jacobi reachability: A brief overview and recent advances. In IEEE Annual Conference on Decision and Control. 2242–2253.

[9] Somil Bansal and Claire J Tomlin. 2021. Deepreach: A deep learning approach to high-dimensional reachability. In 2021 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 1817–1824.

[10] Tamer Başar and Geert Jan Olsder. 1998. Dynamic Noncooperative Game Theory. SIAM.

[11] Stéphane Boucheron, Gábor Lugosi, and Pascal Massart. 2013. Concentration inequalities. Oxford University Press, Oxford.

[12] Siddhartha Chib and Edward Greenberg. 1995. Understanding the metropolis-hastings algorithm. The american statistician 49, 4 (1995), 327–335.

[13] Christian M Chilan and Bruce A Conway. 2020. Optimal nonlinear control using Hamilton–Jacobi–Bellman viscosity solutions on unstructured grids. Journal of Guidance, Control, and Dynamics (2020).

[14] Yinlam Chow, Aviv Tamar, Shie Mannor, and Marco Pavone. 2015. Risk-sensitive and robust decision-making: a cvar optimization approach. Advances in neural information processing systems 28 (2015).

[15] Benjamin A Christie and Dylan P Losey. 2024. LIMIT: Learning interfaces to maximize information transfer. ACM Transactions on Human-Robot Interaction 13, 4 (2024), 1–26.

[16] Anushri Dixit, Lars Lindemann, Skylar X Wei, Matthew Cleaveland, George J Pappas, and Joel W Burdick. 2023. Adaptive conformal prediction for motion planning among dynamic agents. In Learning for Dynamics and Control Conference. PMLR, 300–314.

[17] Franz Faul, Edgar Erdfelder, Albert-Georg Lang, and Axel Buchner. 2007. G* Power 3: A flexible statistical power analysis program for the social, behavioral, and biomedical sciences. Behavior research methods 39, 2 (2007), 175–191.

[18] Jaime F Fisac, Neil F Lugovoy, Vicenç Rubies-Royo, Shromona Ghosh, and Claire J Tomlin. 2019. Bridging hamilton-jacobi safety analysis and reinforcement learning. In 2019 International Conference on Robotics and Automation (ICRA). IEEE, 8550–8556.

[19] David Fridovich-Keil, Andrea Bajcsy, Jaime F Fisac, Sylvia L Herbert, Steven Wang, Anca D Dragan, and Claire J Tomlin. 2020. Confidence-aware motion prediction for real-time collision avoidance. The International Journal of Robotics Research 39, 2-3 (2020), 250–265.

[20] David Fridovich-Keil, Ellis Ratner, Lasse Peters, Anca D Dragan, and Claire J Tomlin. 2020. Efficient iterative linear-quadratic approximations for nonlinear multi-player general-sum differential games. In IEEE International Conference on Robotics and Automation. 1475–1481.

[21] Joshua Hoegerman and Dylan Losey. 2023. Reward learning with intractable normalizing functions. IEEE Robotics and Automation Letters 8, 11 (2023), 7511–7518.

[22] Kai-Chieh Hsu, Haimin Hu, and Jaime F Fisac. 2023. The safety filter: A unified view of safety-critical control in autonomous systems. Annual Review of Control, Robotics, and Autonomous Systems 7 (2023).

[23] Haimin Hu, David Isele, Sangjae Bae, and Jaime F Fisac. 2024. Active uncertainty reduction for safe and efficient interaction planning: A shielding-aware dual control approach. The International Journal of Robotics Research 43, 9 (2024), 1382–1408.

[24] Haimin Hu, Zixu Zhang, Kensuke Nakamura, Andrea Bajcsy, and Jaime F Fisac. 2023. Deception game: Closing the safety-learning loop in interactive robot autonomy. arXiv preprint arXiv:2309.01267 (2023).

[25] Chii-Ruey Hwang. 1980. Laplace's method revisited: weak convergence of probability measures. The Annals of Probability (1980), 1177–1182.

[26] Rufus Isaacs. 1999. Differential games: a mathematical theory with applications to warfare and pursuit, control and optimization. Courier Corporation.

[27] Frank Jiang, Glen Chou, Mo Chen, and Claire J Tomlin. 2016. Using neural networks to compute approximate and guaranteed feasible Hamilton-Jacobi-Bellman PDE solutions. arXiv preprint arXiv:1611.03158 (2016).

[28] Morgan Jones and Matthew M Peet. 2020. Polynomial approximation of value functions and nonlinear controller design with performance bounds. arXiv preprint arXiv:2010.06828 (2020).

[29] Kushal Kedia, Atiksh Bhardwaj, Prithwish Dan, and Sanjiban Choudhury. 2024. Interact: Transformer models for human intent prediction conditioned on robot actions. In 2024 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 621–628.

[30] Solomon Kullback and Richard A Leibler. 1951. On information and sufficiency. The annals of mathematical statistics 22, 1 (1951), 79–86.

[31] Forrest Laine, David Fridovich-Keil, Chih-Yuan Chiu, and Claire Tomlin. 2023. The computation of approximate generalized feedback Nash equilibria. SIAM Journal on Optimization 33, 1 (2023), 294–318.

[32] Lars Lindemann, Matthew Cleaveland, Gihyun Shim, and George J Pappas. 2023. Safe planning in dynamic environments using conformal prediction. IEEE Robotics and Automation Letters 8, 8 (2023), 5116–5123.

[33] Anirudha Majumdar and Marco Pavone. 2019. How should a robot assess risk? towards an axiomatic theory of risk in robotics. In Robotics Research: The 18th International Symposium ISRR. Springer, 75–84.

[34] Tenavi Nakamura-Zimmerer, Qi Gong, and Wei Kang. 2020. A causality-free neural network method for high-dimensional Hamilton-Jacobi-Bellman equations. In American Control Conference. 787–793.

[35] Tenavi Nakamura-Zimmerer, Qi Gong, and Wei Kang. 2021. Adaptive deep learning for high-dimensional Hamilton–Jacobi–Bellman equations. SIAM Journal on Scientific Computing 43, 2 (2021), A1221–A1247.

[36] Youngim Nam and Cheolhyeon Kwon. 2024. Active inference-based planning for safe human-robot interaction: Concurrent consideration of human characteristic and rationality. IEEE robotics and automation letters 9, 8 (2024), 7086–7093.

[37] Sagar Parekh, Lauren Bramblett, Nicola Bezzo, and Dylan P Losey. 2025. Using high-level patterns to estimate how humans predict a robot will behave. In 2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 16947–16954.

[38] Sagar Parekh and Dylan P Losey. 2023. Learning latent representations to co-adapt to humans. Autonomous Robots 47, 6 (2023), 771–796.

[39] Gareth O Roberts and Jeffrey S Rosenthal. 2004. General state space Markov chains and MCMC algorithms. (2004).

[40] R Tyrrell Rockafellar, Stanislav Uryasev, et al. 2000. Optimization of conditional value-at-risk. Journal of risk 2 (2000), 21–42.

[41] Shahabedin Sagheb, Sagar Parekh, Ravi Pandya, Ye-Ji Mun, Katherine Driggs-Campbell, Andrea Bajcsy, and Dylan P Losey. 2025. A unified framework for robots that influence humans over long-term interaction. arXiv preprint arXiv:2503.14633 (2025).

[42] Maurice Sion. 1958. On general minimax theorems. (1958).

[43] Oliver Slumbers, David Henry Mguni, Stefano B Blumberg, Stephen Marcus Mcaleer, Yaodong Yang, and Jun Wang. 2023. A game-theoretic framework for managing risk in multi-agent systems. In International Conference on Machine Learning. 32059–32087.

[44] Alan Wilbor Starr and Yu-Chi Ho. 1969. Nonzero-sum differential games. Journal of Optimization Theory and Applications 3 (1969), 184–206.

[45] Ran Tian, Liting Sun, Andrea Bajcsy, Masayoshi Tomizuka, and Anca D Dragan. 2022. Safety assurances for human-robot interaction via confidence-aware game-theoretic human models. In IEEE International Conference on Robotics and Automation. 11229–11235.

[46] Luke Tierney. 1994. Markov chains for exploring posterior distributions. the Annals of Statistics (1994), 1701–1728.

[47] Pauli Virtanen, Ralf Gommers, Travis E Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. 2020. SciPy 1.0: fundamental algorithms for scientific computing in Python. Nature methods 17, 3 (2020), 261–272.

[48] Kim P Wabersich, Andrew J Taylor, Jason J Choi, Koushil Sreenath, Claire J Tomlin, Aaron D Ames, and Melanie N Zeilinger. 2023. Data-driven safety filters: Hamilton-jacobi reachability, control barrier functions, and predictive methods for uncertain systems. IEEE Control Systems Magazine 43, 5 (2023), 137–177.

[49] Kim P Wabersich and Melanie N Zeilinger. 2018. Linear model predictive safety certification for learning-based control. In 2018 IEEE Conference on Decision and Control (CDC). IEEE, 7130–7135.

[50] Kim P Wabersich and Melanie N Zeilinger. 2021. A predictive safety filter for learning-based control of constrained nonlinear dynamical systems. Automatica 129 (2021), 109597.

[51] Mingyu Wang, Negar Mehr, Adrien Gaidon, and Mac Schwager. 2020. Game-theoretic planning for risk-aware interactive agents. In IEEE/RSJ International Conference on Intelligent Robots and Systems.
[52] Jiduan Wu, Anas Barakat, Ilyas Fatkhullin, and Niao He. 2023. Learning zero-sum linear quadratic games with improved sample complexity. In IEEE Conference on Decision and Control. 2602–2609.

A DERIVATIONS OF SECTION 4.4

Here we fill in the details behind the convergence analysis in Section 4.4. Each subsection corresponds to one of the three sources of error. In Appendix A.1 we bound how concentrated the target distributions are (*Finite β*). In Appendix A.2 we explain why the inner and outer Metropolis-Hastings samplers approach their targets exponentially quickly (*Finite M*). Then, in Appendix A.3 we use these results to derive the nested sampler error $\varepsilon_{N,\beta}$ (*Finite N*).

Throughout, let $\mathcal{K}_u$ be the set of robot trajectories that the outer loop can propose, and let $\mathcal{K}_w$ be the set of human trajectories that the inner loop can propose. The robot's set is $\mathcal{K}_u = \mathcal{U}^T$. The human's set is $\mathcal{K}_w = \mathcal{W}^T$, intersected with the ball of radius $\sqrt{\lambda}$ around the nominal prediction when the safety margin in Equation (19) is used. Both sets are compact and convex, because $\mathcal{U}$ and $\mathcal{W}$ are compact and convex (Section 3) and the ball is convex. To measure the difference between two distributions P and Q, we use the total variation distance $\|P - Q\|_{\mathrm{TV}} = \sup_A |P(A) - Q(A)|$.

A.1 Concentration of the Target Distributions

Here we derive the first term of Equation (24) and the simplified bounds for linear-quadratic games. The same argument applies to both loops, so we state it for a generic target $\pi(\xi) \propto \exp(-\beta U(\xi))$ over a compact, convex set $\mathcal{K}$ with diameter D, where each trajectory has d elements. The outer loop uses $U = F$ and $\mathcal{K} = \mathcal{K}_u$; the inner loop uses $U = -J$ and $\mathcal{K} = \mathcal{K}_w$. Let $\xi^\star$ minimize U, and let $\Delta(r) = \inf_{\|\xi - \xi^\star\| > r} [U(\xi) - U(\xi^\star)]$ be the optimality gap at radius r.

Assumptions. For this analysis, we only require that U changes at a bounded rate: $|U(\xi) - U(\xi')| \le G\|\xi - \xi'\|$ for some constant G. This condition follows directly from Section 3: the gradient of J with respect to ξ_w is continuous over a compact set, and so it is bounded by some G_w. Likewise for the outer loop: the gradient of J with respect to ξ_u is bounded for some G_u. Every human trajectory therefore gives a robot cost $J(x, \cdot, \xi_w)$ that changes at a rate of at most G_u, so the worst case F does as well. Note that F does not need to be smooth.

A Bound on the Tails. A key reason to use Metropolis-Hastings in the first place is that we do not need the normalizing constant of π [21]. Instead, we only use the ratio $\pi(\xi)/\pi(\xi^\star) = \exp(-\beta(U(\xi) - U(\xi^\star)))$, which is the same ratio that the sampler uses to accept or reject proposals. Every trajectory farther than r from $\xi^\star$ has density at most $\pi(\xi^\star)\exp(-\beta\Delta(r))$, and so together these trajectories have probability at most $\mathrm{vol}(\mathcal{K})\,\pi(\xi^\star)\exp(-\beta\Delta(r))$. It remains to bound the density at the optimum, $\pi(\xi^\star)$. Because $\mathcal{K}$ is convex, it contains a copy of itself shrunk towards $\xi^\star$ by a factor δ/D. This copy has volume $(\delta/D)^d\,\mathrm{vol}(\mathcal{K})$, and every trajectory in it lies within δ of $\xi^\star$, so its density is at least $\pi(\xi^\star)\exp(-\beta G\delta)$. The copy cannot have probability greater than one, and so $\pi(\xi^\star) \le \exp(\beta G\delta)/\big((\delta/D)^d\,\mathrm{vol}(\mathcal{K})\big)$. Combining these two results and choosing $\delta = 1/\beta$:

$$\pi\big(\|\xi - \xi^\star\| > r\big) \le C\exp\left(-\beta\Delta(r)\right), \qquad C = (\beta D)^d\exp(G) \tag{A.1}$$

For the outer loop this gives the first term of Equation (24) with $C_1 = (\beta D_u)^d\exp(G_u)$, where D_u is the diameter of $\mathcal{K}_u$. The constant C grows only polynomially in β, while $\exp(-\beta\Delta(r))$ decays exponentially; so for any radius with $\Delta(r) > 0$, the bound vanishes as β increases.

Linear-Quadratic Games. When F is quadratic with Hessian $H \succeq \mu I$ and we ignore the action bounds, P_β is a Gaussian with mean $\xi_u^\star$ and covariance $(\beta H)^{-1}$, which is no larger than $(\beta\mu)^{-1}I$. Each element of the trajectory is therefore a Gaussian with standard deviation at most $1/\sqrt{\beta\mu}$. The standard Gaussian tail bound gives the first term of Equation (25). For the full trajectory, the expected distance from the robust trajectory is at most $\sqrt{d/(\beta\mu)}$, and the distance concentrates

around this value [11]. For $r > \sqrt{d/(\beta\mu)}$:

$$P_\beta\left(\|\xi_u - \xi_u^\star\| > r\right) \leq \exp\left(-\frac{\beta\mu}{2}\left(r - \sqrt{\frac{d}{\beta\mu}}\right)^2\right) \tag{A.2}$$

The same bound holds for the executed action with d replaced by m.

A.2 Uniform Ergodicity of the Samplers

The inner loop of MCLQ is a Metropolis-Hastings sampler with target $\pi_w(\xi_w) \propto \exp(\beta J(x, \xi_u, \xi_w))$ over $\mathcal{K}_w$, with the robot's trajectory ξ_u held fixed. The idealized outer loop is a Metropolis-Hastings sampler with target $P_\beta(\xi_u) \propto \exp(-\beta F(\xi_u))$ over $\mathcal{K}_u$. As in Appendix A.1, we state our result for a generic target $\pi(\xi) \propto \exp(-\beta U(\xi))$ over a compact set $\mathcal{K}$. Metropolis-Hastings samplers are known to converge to their targets. Here we need two stronger properties: the convergence is exponentially fast, and it holds from *every* starting trajectory. Both properties follow from a standard result once we show that every transition of the sampler has some chance of reaching anywhere.

A Lower Bound on Every Transition. Since J is continuous over a compact set, it is bounded; let $\Omega = \max J - \min J$ denote its range. Since $F(\xi_u) = \sup_{\xi_w} J(x, \xi_u, \xi_w)$ lies between the same bounds, the range of U is at most Ω for both loops. A proposal ξ' that remains in $\mathcal{K}$ is therefore accepted with probability $\min(1, \pi(\xi')/\pi(\xi)) \geq \exp(-\beta\Omega)$. Next, because the perturbations can reach any trajectory in a finite number of steps (Section 4.4), there is a number of steps k and a constant $q > 0$ such that, from any starting trajectory, the density of the k-th proposed trajectory is at least q everywhere in $\mathcal{K}$. For example, if each perturbation is drawn uniformly from a ball of radius ρ, then k is roughly the diameter of $\mathcal{K}$ divided by ρ. A run of k consecutive proposals is all accepted with probability at least $\exp(-k\beta\Omega)$, and so from *any* starting trajectory ξ the probability that the sampler lies in a set $A \subseteq \mathcal{K}$ after k iterations is at least:

$$P^k(\xi, A) \geq \kappa \frac{\mathrm{vol}(A)}{\mathrm{vol}(\mathcal{K})}, \qquad \kappa = q\,\mathrm{vol}(\mathcal{K})\exp\left(-k\beta\Omega\right) \tag{A.3}$$

where $P^k(\xi, \cdot)$ is the distribution of the sampler after k iterations. Put another way, every block of k iterations contains (with probability at least κ) a uniformly random trajectory that does not depend on where the sampler began.

Convergence. Equation (A.3) is known as a Doeblin or minorization condition. Theorem 8 of [39] shows that any sampler that satisfies it is uniformly ergodic: after n iterations, and from every starting trajectory ξ,

$$\|P^n(\xi, \cdot) - \pi\|_{\mathrm{TV}} \leq (1-\kappa)^{\lfloor n/k \rfloor} \leq \frac{\exp\left(-c\,n\right)}{1-\kappa}, \qquad c = \frac{-\ln(1-\kappa)}{k} \tag{A.4}$$

Intuitively, once a fresh start has occurred, the sampler has forgotten where it began.

Consequences. Equation (A.4) provides the rates used in Section 4.4. Applied to the outer loop after M iterations, it gives the term $C_2\exp(-cM)$ in Equation (24), with $C_2 = 1/(1-\kappa)$. Applied to the inner loop after N iterations, it gives $\|P^N(\xi_w, \cdot) - \pi_w\|_{\mathrm{TV}} \leq C_2'\exp(-c'N)$, which we use below. Two features of Equation (A.4) matter for what follows. First, the bound holds from every starting trajectory, so it does not matter how the sampler was initialized; this includes the warm starts used by MCLQ. Second, the constants are worst-case: κ shrinks exponentially as β or k grows, so larger inverse temperatures and smaller perturbations slow the guaranteed rate of convergence. This is the formal version of the trade-off noted in Section 4.4. It also means that Equation (A.4) is conservative: in practice the samplers converge much faster than these constants suggest.

A.3 Nested Sampler Error

Here we derive the term $\varepsilon_{N,\beta}$ from Equation (24). MCLQ's outer loop scores each robot proposal against the human trajectory $\tilde{\xi}_w$ currently held by the inner loop (line 22 of Algorithm 1), while an *idealized* outer loop would score it against a worst-case response $\xi_w^\star(\xi_u) \in \arg\max_{\xi_w \in \mathcal{K}_w} J(x, \xi_u, \xi_w)$. We bound how often these two loops make different decisions. We do not assume that J is concave in the human's actions, so the human may have several worst-case responses (e.g., approaching the robot from either side); $\xi_w^\star(\xi_u)$ denotes any one of them, and every step below holds for any such choice.

Assumptions. In addition to the assumptions in Section 4.4, our analysis of the nested sampler relies on two assumptions:

(1) *Bounded coupling between the agents.* The mixed second derivative of J is bounded over $\mathcal{K}_u \times \mathcal{K}_w$: $\|\partial^2 J / \partial \xi_u \partial \xi_w\| \le L_{uw}$.
(2) *Bounded outer perturbations.* Each outer perturbation has size at most ρ, i.e., $\|\xi_u' - \xi_u\| \le \rho$.

Only the nested error $\varepsilon_{N,\beta}$ uses these two assumptions.

Both assumptions are mild. The first follows from Section 3: the cost is computed by rolling out twice continuously differentiable dynamics, so its mixed second derivative is continuous over a compact set and therefore bounded. Intuitively, L_{uw} measures how strongly the robot's plan changes the human's incentives:

$$\left\|\nabla_{\xi_w} J(x, \xi_u', \xi_w) - \nabla_{\xi_w} J(x, \xi_u, \xi_w)\right\| \le L_{uw} \|\xi_u' - \xi_u\| \tag{A.5}$$

For collision avoidance, the mixed derivative comes from the proximity cost, and if the robot's cost did not depend on the human at all, then $L_{uw} = 0$. The second assumption is trivially satisfied by restricting `Perturb` in Algorithm 1, which we do in our implementation.

The Human's Optimality Gap. Similar to the robot's optimality gap in Section 4.4, let $\Delta_w(r)$ be the human's optimality gap at radius r:

$$\Delta_w(r) = \inf_{\xi_u \in \mathcal{K}_u} \inf_{\|\xi_w - \xi_w^\star(\xi_u)\| > r} \left[F(\xi_u) - J(x, \xi_u, \xi_w)\right] \tag{A.6}$$

Put another way, every human trajectory farther than r from the worst case is less harmful to the robot by at least $\Delta_w(r)$. When the worst case is distinct, $\Delta_w(r)$ grows with r. When two very different human trajectories are almost equally harmful, $\Delta_w(r)$ is close to zero until r exceeds the distance between them. No assumption is placed on Δ_w: the results below hold for any cost.

Setup. We run MCLQ and the idealized outer loop side by side. They start from the same robot trajectory and use the same proposals and the same uniform random numbers. At each iteration the two loops make the same decision unless the uniform random number falls between their two acceptance probabilities. The probability of a disagreement at any single iteration is therefore at most the difference between the acceptance probabilities. Since $\min(1, \exp(-\beta z))$ changes by at most β times any change in z, this difference is at most β times the error in the scored cost difference. Until the first disagreement, both loops are at the same robot trajectory.

One Iteration. Consider an outer iteration in which both loops are at ξ_u and propose ξ_u', and let $a = \xi_w^\star(\xi_u)$ and $b = \xi_w^\star(\xi_u')$ be the worst-case responses to the current and proposed robot trajectories. Let

$$g(\xi_w) = J(x, \xi_u', \xi_w) - J(x, \xi_u, \xi_w)$$

be the cost difference between the proposal and the current trajectory for a fixed human trajectory. MCLQ scores the proposal with $g(\tilde{\xi}_w)$, while the idealized loop uses $F(\xi_u') - F(\xi_u)$. Because each

worst-case response maximizes J, the exact cost difference lies between g evaluated at the two worst-case responses:

$$g(a) \leq F(\xi_u') - F(\xi_u) \leq g(b) \tag{A.7}$$

For the upper bound, $F(\xi_u) = J(x, \xi_u, a) \geq J(x, \xi_u, b)$; the lower bound follows in the same way at ξ_u'. Note that Equation (A.7) does not require a or b to be unique. The gradient of g is the difference of two gradients at robot trajectories at most ρ apart, so by Equation (A.5) g changes by at most $L_{uw}\rho$ per unit change in ξ_w. Since the exact cost difference lies between $g(a)$ and $g(b)$, the error in MCLQ's score is at most:

$$\left|g(\tilde{\xi}_w) - \left[F(\xi_u') - F(\xi_u)\right]\right| \leq L_{uw}\rho \max\left\{\|\tilde{\xi}_w - a\|,\ \|\tilde{\xi}_w - b\|\right\} \leq L_{uw}\rho\Big(\|\tilde{\xi}_w - a\| + \|a - b\|\Big) \tag{A.8}$$

The first term is how far the inner loop is from the current worst case. The second term is how far the worst case moves when the robot changes its trajectory by ρ. Since both $\tilde{\xi}_w$ and b lie in $\mathcal{K}_w$, the middle expression of Equation (A.8) is never more than $L_{uw}\rho D_w$, where D_w is the diameter of $\mathcal{K}_w$. We now bound the two terms more precisely.

How Far the Worst Case Moves. Without concavity, the worst case can jump: a small change in the robot's plan can switch the human from one harmful response to another. However, the new worst case b cannot be much worse than a for the *current* robot trajectory. Consider $h(\xi_u) = J(x, \xi_u, b) - J(x, \xi_u, a)$. We know $h(\xi_u') \geq 0$ because b is the worst case for ξ_u', and by Equation (A.5) h changes by at most $L_{uw}\|b - a\|$ per unit change in ξ_u. Therefore:

$$F(\xi_u) - J(x, \xi_u, b) = -h(\xi_u) \leq L_{uw}\rho\|b - a\| \leq L_{uw}\rho D_w \tag{A.9}$$

In words, b is nearly a worst case for the current robot trajectory. Comparing Equation (A.9) with Equation (A.6): for any radius r with $\Delta_w(r) > L_{uw}\rho D_w$, the worst case moves by at most $\|a - b\| \leq r$.

Accuracy of the Inner Loop. This error has two parts: the inner loop's target $\pi_w \propto \exp(\beta J)$ is a distribution rather than a single point, and after N iterations the inner loop may not have reached that target.

For the first part, we apply Equation (A.1) to the inner loop, with $U = -J$, $\mathcal{K} = \mathcal{K}_w$, and $G = G_w$. Every human trajectory farther than r from a receives at most $\exp(-\beta\Delta_w(r))$ times the probability of the worst case, and so:

$$\pi_w\Big(\|\xi_w - a\| > r\Big) \leq C_w \exp\left(-\beta\Delta_w(r)\right), \qquad C_w = (\beta D_w)^{d_w} \exp(G_w) \tag{A.10}$$

where d_w is the number of elements in the human's trajectory (i.e., the dimension of $\mathcal{W}$ times the control horizon). Like C_1, C_w grows only polynomially in β. Because $\|\xi_w - a\|$ is at most r on this event and at most D_w otherwise, samples from the target satisfy:

$$\mathbb{E}_{\xi_w \sim \pi_w}\Big[\|\xi_w - a\|\Big] \leq r + D_w C_w \exp\left(-\beta\Delta_w(r)\right) \tag{A.11}$$

For the second part, Appendix A.2 shows that after N iterations the inner loop's distribution is within $C_2' \exp(-c'N)$ of π_w in total variation, from any starting trajectory; this result does not require concavity either. The distance $\|\xi_w - a\|$ always lies between 0 and D_w. For a quantity confined to $[0, D_w]$, the expectations under two distributions differ by at most D_w times their total variation distance [39, Proposition 3(b)]. Therefore:

$$\mathbb{E}\Big[\|\tilde{\xi}_w - a\|\Big] \leq r + D_w C_w \exp\left(-\beta\Delta_w(r)\right) + D_w C_2' \exp\left(-c'N\right) \tag{A.12}$$

The Sequential Dependency. Algorithm 1 warm-starts the inner loop: the human's trajectory is never reset between outer iterations, so the inner loop at each outer iteration begins wherever it stopped at the previous one. At that point it was searching for the worst case of a different

robot trajectory. This is the dependency between the inner and outer samplers that prevents a direct application of single-loop results. Equation (A.4) resolves it. The bound holds from *every* starting trajectory, so Equation (A.12) holds conditioned on everything that happened earlier in the search. The only fact we use is that the inner loop runs N iterations while the robot's trajectory is held fixed at the current ξ_u (lines 13–20). Substituting Equation (A.9) and Equation (A.12) into Equation (A.8) and multiplying by β, the probability that the two loops disagree at a given iteration, conditioned on the history of the search, is at most:

$$\epsilon_{\text{step}} = \beta L_{uw}\rho \ \min\left\{D_w,\ 2r + D_w C_w \exp\left(-\beta\Delta_w(r)\right) + D_w C_2' \exp\left(-c'N\right)\right\} \tag{A.13}$$

for any radius r with $\Delta_w(r) > L_{uw}\rho D_w$. If no such radius exists, the first entry of the minimum still applies.

Result. The two loops start together and can only separate after a disagreement. Conditioned on the history, each of the M outer iterations produces the first disagreement with probability at most ϵ_{step}, so the probability that the loops ever disagree is at most $M\epsilon_{\text{step}}$. If they never disagree, MCLQ returns exactly the trajectory that the idealized loop would have returned. The probability of any event, in particular $\|\xi_u' - \xi_u^\star\| \le r$, therefore differs between the two loops by at most $M\epsilon_{\text{step}}$, by the coupling inequality [39, Section 4.1]. The idealized loop satisfies the first two terms of Equation (24), so Equation (24) holds with:

$$\varepsilon_{N,\beta} = M\beta L_{uw}\rho \ \min\left\{D_w,\ 2r + D_w C_w \exp\left(-\beta\Delta_w(r)\right) + D_w C_2' \exp\left(-c'N\right)\right\} \tag{A.14}$$

This expression makes the properties in Section 4.4 explicit. Every term in Equation (A.14) is proportional to L_{uw} and to ρ, and the last term decays exponentially in N. The remaining terms depend on how distinct the human's worst case is: if two human responses are almost equally harmful, $\Delta_w(r)$ is small and the bound falls back to $M\beta L_{uw}\rho D_w$. This matches intuition, since the inner loop may then settle on either response, and no number of inner iterations can tell the robot which one the exact worst case will use after the robot changes its plan.

Practical Notes. First, the perturbation size ρ presents a trade-off: smaller outer perturbations reduce $\varepsilon_{N,\beta}$, but they increase the number of steps k needed to reach any trajectory, and therefore slow the outer loop's mixing in Equation (A.4). Second, this bound is conservative. It treats every error in the scored cost as if it changed a decision, and it counts a disagreement early in the search as if it were never forgotten. In a scalar linear-quadratic game with a known solution, we found that the error caused by the nested structure was much smaller than Equation (A.14). It decreased with N, increased with ρ, and was indistinguishable from zero when the robot and human were decoupled ($L_{uw} = 0$).

B IMPLEMENTATION DETAILS

Here we list the parameters of MCLQ used in each experiment. An implementation of MCLQ, including the configuration files for these experiments, is available at https://github.com/safe-interactions-mclq/mclq.

Parameters. Table 1 lists the parameters for each experiment. The *planning horizon* is the number of timesteps in each open-loop problem (Section 4.4). The inner loop is parallelized across threads: the number of inner iterations N is the total across all threads.

Selection. We selected these values in small pilot runs. For the numbers of iterations, we chose the smallest values that let MCLQ plan in real time for each environment. For the inverse temperature, larger values concentrate the target distribution around the robust trajectory (Appendix A.1) but slow the guaranteed rate of convergence (Appendix A.2); we chose values of β that balanced these

Table 1. Parameters of MCLQ and of the simulated humans in each experiment.

	Point-Mass	Driving	Manipulator
Planning horizon (timesteps)	10	10	10
Outer iterations M	1000	800	300
Inner iterations N (all threads)	600	600	600
Threads	12	12	12
Inverse temperature β	10	5	5
Perturbation scale	0.05	0.1	0.05
Human rationality α	7.5	7.5	7.5

effects and avoided numerical issues in the acceptance ratio. The perturbation scale presents a similar trade-off: smaller perturbations reduce the nested sampler error but slow mixing (Appendix A.3). The rationality of the simulated humans matches Section 5, and the user study's nominal human model is given in Equation (31) with $\beta_H = 5.0$.